



Fachhochschul-Bachelorstudiengang

SOFTWARE ENGINEERING

A-4232 Hagenberg, Austria

Performancevergleich zwischen JavaScript und WebAssembly anhand eines clientseitigen Backtesters

Bachelorarbeit

zur Erlangung des akademischen Grades
Bachelor of Science in Engineering

Eingereicht von

Florian Rittberger

Begutachtet von Andreas Johann Scheibenpflug, BSc MSc

Hagenberg, Februar 2026

Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen entnommenen Stellen als solche gekennzeichnet habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt.

Die vorliegende, gedruckte Bachelorarbeit ist identisch zu dem elektronisch übermittelten Textdokument.

Datum: 24.01.2026

Unterschrift: 

Kurzfassung

In dieser Arbeit wird die Performanz von WebAssembly und JavaScript verglichen. Der Vergleich wird mit einem clientseitigen Backtester durchgeführt, der Handelsstrategien in beiden Sprachen unterstützt. Ein Backtester ist eine Software, mit der Strategien anhand historischer Kursdaten getestet werden können. Handelsstrategien sind Algorithmen, die automatisch Kauf- und Verkaufsentscheidungen treffen, um Gewinne zu erzielen, ohne selbst aktiv handeln zu müssen. Der Backtester ist ein geeignetes Programm zum Vergleich der beiden Programmiersprachen, da der Prozess des Backtestings rechenintensiv ist und somit die Performanzunterschiede gut erkennbar sind.

Der Backtester wird als Webanwendung in JavaScript implementiert, die im Browser ausgeführt wird. Für den Vergleich der Sprachen wird der rechenaufwendigste Teil des Backtestings, die Strategien, ausgelagert und in WebAssembly und JavaScript implementiert. Damit die Ausführungszeiten bei unterschiedlichem Rechenaufwand verglichen werden können, werden drei unterschiedliche Handelsstrategien mit steigender Komplexität implementiert. Die entwickelten Strategien sind eine Gleitende Durchschnitts Überscheidungsstrategie, eine Ausbruchsstrategie und eine Mittelwertrückkehrstrategie mit Bollinger Bänder. Anhand dieser Strategien kann die Performanz und Varianz von JavaScript und WebAssembly verglichen werden.

Insgesamt ist zu erkennen, dass die Performanz von WebAssembly und JavaScript vom verwendeten Browser und der Rechenintensität der Strategie abhängt. Jedoch kann WebAssembly bei rechenintensiven Aufgaben einen großen Vorteil in der Ausführung bieten. Durch die Tests der Strategien mit verschiedenen Datensätzen ist erkennbar, dass WebAssembly in Firefox bei rechenintensiven Strategien schneller ist als JavaScript. Ebenfalls ist die Varianz der durchschnittlichen Ausführungszeiten von WebAssembly in Firefox geringer als die von JavaScript. In Chrome hingegen ist JavaScript bei jeder getesteten Strategie schneller als WebAssembly. Die Varianz ist in Chrome bei WebAssembly geringer als bei JavaScript.

Abstract

This paper compares the performance of WebAssembly and JavaScript. The comparison is made with a clientside backtester that supports trading strategies in both languages. A backtester is a software that allows the testing of trading strategies based on historical price data. Strategies are algorithms that automatically make buying and selling decisions in order to make profits without active interaction. The backtester is a suitable program for comparing the two programming languages, because the backtesting process is very computationally intensive and thus the performance differences are clearly visible.

The backtester is implemented as a web application in JavaScript that runs in the browser. To compare the languages, the most computationally intensive part of backtesting, the strategies, is outsourced and implemented in WebAssembly and JavaScript. To be able to compare the execution times at different computational loads, three different trading strategies with increasing complexity are implemented. The developed strategies are a moving average crossover strategy, a breakout strategy and a mean reversion strategy with Bollinger bands. Because of these strategies, the performance and variance of JavaScript and WebAssembly can be compared.

Testing the strategies with different data sets shows that WebAssembly in Firefox is faster than JavaScript for computationally intensive strategies. Furthermore, the variance in the average execution times for WebAssembly in Firefox is lower than that for JavaScript. In Chrome, on the other hand, JavaScript is faster than WebAssembly for every strategy tested. The variance in Chrome is lower for WebAssembly than for JavaScript. Overall, it can be seen that the performance of WebAssembly and JavaScript depends on the browser used and the computational intensity of the strategy. However, WebAssembly can offer a significant advantage in execution for computationally intensive tasks.

Inhaltsverzeichnis

Kurzfassung	i
Abstract	ii
1 Einleitung	1
1.1 Ziele und Motivation der Arbeit	2
1.2 Vorgangsweise zur Zielerreichung	2
1.3 Kapitelübersicht	2
1.3.1 Grundlegende Konzepte und Technologien	2
1.3.2 Konzept und Design	2
1.3.3 Implementierung des Backtesters und der Strategien	3
1.3.4 Performanztests der unterschiedlichen Programmiersprachen	3
1.3.5 Zusammenfassung und Schlussbemerkungen	3
2 Grundlegende Konzepte und Technologien	4
2.1 Backtesting	4
2.2 Biase in Backtesting	6
2.2.1 Look-Ahead Bias	6
2.2.2 Overfitting Bias	6
2.2.3 Survivorship Bias	6
2.2.4 Data-Snooping Bias	7
2.3 State of the Art Backtester	7
2.3.1 Backtrader	7
2.3.2 Zipline	8
2.3.3 BacktestJS	8
2.4 Bewährte Ansätze für Handelsstrategien	9
2.4.1 Mittelwertrückkehr mit Bollinger Bänder	9
2.4.2 Gleitende Durchschnitts Überscheidungen	11
2.4.3 Ausbruchsstrategien	11
2.5 WASM	12
2.5.1 Ressourcenverbrauch und Performanz von WASM	13
2.6 Laufzeitumgebungen für WebAssembly	13
2.6.1 Ausführung der WebAssembly-Anweisungen	14
3 Konzept und Design	15
3.1 Architektur des Backtesters	15

3.1.1	Format der historischen Finanzdaten	16
3.1.2	Backtesterlogik	16
3.1.3	Modularisierung der Strategien	17
3.2	Aufbau der Teststrategien	18
3.2.1	Struktur der Mittelwertrückkehr Strategie mit Bollinger Bändern	19
3.2.2	Aufbau der Ausbruchsstrategie	19
3.2.3	Umsetzung der gleitenden Durchschnittsüberschneidungs Strategie	19
3.3	Weboberfläche des Backtesters	20
3.4	Visualisierung der Daten und Ergebnisse	21
3.5	Aufbau der Geschwindigkeitstests	21
3.6	Abrufen der historischen Kursdaten	22
3.7	Werkzeuge zur WASM Entwicklung	22
3.7.1	Emscripten	22
3.7.2	WABT	23
4	Implementierung des Backtesters und der Strategien	24
4.1	Die Benutzerschnittstellen des Backtesters	24
4.2	Laden und Vorverarbeitung der Kursdaten	25
4.3	Laden der Strategien	26
4.4	Implementierung der Backtestlogik	27
4.5	Implementierungen der Strategien	28
4.5.1	Gleitende Durchschnittsüberschneidungs Strategie	29
4.5.2	Mittelwertrückkehr Strategie	30
4.5.3	Ausbruchsstrategie	32
4.6	Die Datenvisualisierungen	33
4.6.1	Visualisierung des Kurswertes	33
4.6.2	Visualisierung des Portfoliowertes	33
5	Performanztests der unterschiedlichen Programmiersprachen	35
5.1	Die Testvoraussetzungen	35
5.1.1	Die Testumgebung	35
5.1.2	Die Testdaten	35
5.2	Die Testausführung	36
5.3	Analyse der Testergebnisse	37
5.3.1	Ergebnisse in Firefox	38
5.3.2	Ergebnisse in Chrome	41
5.3.3	Browserabhängige Performanz von WebAssembly im Vergleich .	44
5.3.4	Analyse der WebAssembly Performanz ohne Berücksichtigung der Speicherverwaltung	44
5.4	Zusammenfassung der Testergebnisse	46
6	Zusammenfassung und Schlussbemerkungen	48
6.1	Zusammenfassung	48
6.2	Ausblick und mögliche Erweiterungen des Backtesters	49
	Quellenverzeichnis	50
	Literatur	50

Inhaltsverzeichnis	v
Software	53

Kapitel 1

Einleitung

Automatisiertes Handeln von Wertpapieren ist heute ein wesentlicher Bestandteil vieler Finanzmarktanwendungen. [28] Die Grundlage für einen solchen Handel sind Handelsstrategien, die verschiedenste Algorithmen implementierten und ohne manuelles Eingreifen ausgeführt werden können. Um die Wirtschaftlichkeit und Stabilität einer Strategie zu bewerten, ist ein systematisches Testen auf Basis von historischer Finanzdaten notwendig. Für diese Validierung werden Backtester eingesetzt.

Backtester sind Softwarewerkzeuge, mit denen das Verhalten einer Handelsstrategie unter historischen Marktbedingungen simuliert werden kann. Dadurch können Strategien vor dem Einsatz im echten Finanzmarkt bewertet und optimiert werden. In vielen bestehenden Lösungen erfolgt das Backtesting serverseitig oder über externe Drittanbieter, wodurch Handelsstrategien an fremde Systeme übergeben werden müssen. Die Verwendung externer Infrastrukturen zur Ausführung von Strategien hat jedoch Risiken hinsichtlich des Datenschutzes und der Vertrauenswürdigkeit der Ausführungsumgebung. Es kann nicht ausgeschlossen werden, dass sensible Informationen über die Strategien abgefangen oder missbraucht werden. Mit einem clientseitigen Backtester kann dieses Risiko vermieden werden, da die Strategien lokal im Browser ausgeführt werden und nicht an externe Systeme weitergegeben werden müssen. Allerdings hat die clientseitige Ausführung des Backtesters auch den Nachteil, dass die Performanz im Vergleich zu den serverseitigen Lösungen geringer sein kann. Moderne Webtechnologien ermöglichen jedoch zunehmend, dass auch aufwendige Berechnungen auf der Clientseite performant und stabil durchgeführt werden können. Insbesondere WebAssembly bietet immer mehr neue Möglichkeiten, rechenintensive Logik effizient clientseitig im Web auszuführen.

In dieser Arbeit wird ein clientseitiger Backtester entwickelt, bei dem die Handelsstrategien in WebAssembly und JavaScript ausgeführt werden können. Mit diesem wird die Ausführungsgeschwindigkeit und Stabilität von WebAssembly gegenüber reinem JavaScript getestet und analysiert.

1.1 Ziele und Motivation der Arbeit

Ziel dieser Arbeit ist es, einen Vergleich der Ausführungszeiten von WebAssembly und JavaScript mithilfe eines clientseitigen Backtesters durchzuführen. Da moderne Webanwendungen immer komplexer und rechenintensiver werden, spielt die Performanz der eingesetzten Webtechnologien eine wichtige Rolle. Mit dem Vergleich anhand eines Backtesters soll getestet werden, ob WebAssembly in diesem Anwendungsfall eine höhere Ausführungsgeschwindigkeit als JavaScript erzielen kann. Der Vergleich soll dabei unterschiedlich komplexe Handelsstrategien, verschiedene Datensätze und verschiedene Browserumgebungen umfassen, um die Performanzunterschiede umfassend zu analysieren. Backtesting ist für einen Geschwindigkeitsvergleich geeignet, da es sich um eine rechenintensive Aufgabe handelt, bei der große Mengen historischer Daten von Handelsstrategien verarbeitet werden müssen.

1.2 Vorgangsweise zur Zielerreichung

Damit ein Vergleich der Ausführungszeiten durchgeführt werden kann, wird zuerst ein clientseitiger Backtester entwickelt. Dieser besitzt die Möglichkeit, in JavaScript und WebAssembly implementierte Strategien zu laden und auszuführen.

Die WebAssembly-Strategien werden in C implementiert und anschließend in WebAssembly mit dem Emscripten-Compiler übersetzt. Diese Strategien werden mit unterschiedlichen Algorithmen und Programmiersprachen implementiert, um die Performanzunterschiede bei verschiedenen Komplexitätsgraden zu analysieren. Durch den unterschiedlich hohen Rechenaufwand der Strategien wird der Vergleich umfangreicher und aussagekräftiger.

Die historischen Marktdaten werden von einer externen Quelle bezogen und im Backtester geladen. Der Backtester führt die Strategien mehrfach aus, um die Performanzunterschiede zwischen den Programmiersprachen genauer zu messen. Dadurch wird ein Vergleich der Ausführungszeiten von WebAssembly und JavaScript möglich.

1.3 Kapitelübersicht

1.3.1 Grundlegende Konzepte und Technologien

In dem Kapitel 2 werden die Grundlagen und die relevanten Technologien beschrieben. Dabei werden die grundlegenden Themen wie Backtesting, Biase im Backtesting, bereits existierende Backtester sowie Ansätze für Handelsstrategien näher erläutert. Zudem wird WebAssembly als eine der Kerntechnologien vorgestellt und wie diese in Webanwendungen integriert wird.

1.3.2 Konzept und Design

Das Kapitel 3 beschreibt wie die einzelnen Komponenten des entwickelten Backtester aufgebaut sind, wie die Geschwindigkeitstests durchgeführt werden und welche Werkzeuge für die Entwicklung von WebAssembly genutzt wurden.

1.3.3 Implementierung des Backtesters und der Strategien

In dem Kapitel 4 wird die Implementierung des Backtesters und der Handelsstrategien beschrieben. Dabei wird der gesamte Prozess von der Beschaffung historischer Daten über das Laden und Anwenden der Handelsstrategien bis hin zur Darstellung der Ergebnisse auf der Webseite beschrieben.

1.3.4 Performanztests der unterschiedlichen Programmiersprachen

Das Kapitel 5 zeigt den Aufbau und die Ergebnisse der Performanztests der verschiedenen Implementierungen der Handelsstrategien in unterschiedlichen Programmiersprachen. Es werden Strategien in verschiedene Programmiersprachen implementiert und deren Ausführungszeiten im Backtester in verschiedenen Browsern gemessen. Desweiteren werden die Ergebnisse der Performanztests analysiert und interpretiert.

1.3.5 Zusammenfassung und Schlussbemerkungen

Im Kapitel 6 wird die Arbeit zusammengefasst und ein Ausblick auf mögliche zukünftige Arbeiten gegeben.

Kapitel 2

Grundlegende Konzepte und Technologien

Dieses Kapitel behandelt die theoretischen und technischen Grundlagen des Backtestings. Dabei werden zentrale Konzepte, mögliche Verzerrungen, bewährte Handelsstrategien, bestehende Backtester sowie weitere Technologien wie WebAssembly erläutert.

2.1 Backtesting

Backtesting bietet die Möglichkeit den Erfolg einer Investitionsstrategie an historischen Finanzdaten zu testen. Die Erfolgsquote dieser Strategie lässt sich somit einschätzen, ohne dafür spekulativ Geld in aktuelle Märkte investieren zu müssen. Dabei wird eine Strategie zu konkreten Zeitpunkten in gewissen Intervallen befragt, ob in diesem Moment gekauft, verkauft oder gehalten werden soll. Die Strategie erhält aber für die Berechnung der Entscheidung nur Informationen, die zu diesem Zeitpunkt auch tatsächlich verfügbar gewesen wären. Zukünftige Informationen, die zu diesem Zeitpunkt noch nicht bekannt waren, dürfen nicht in die Entscheidungsfindung mit einfließen, obwohl diese verfügbar sind. [6]

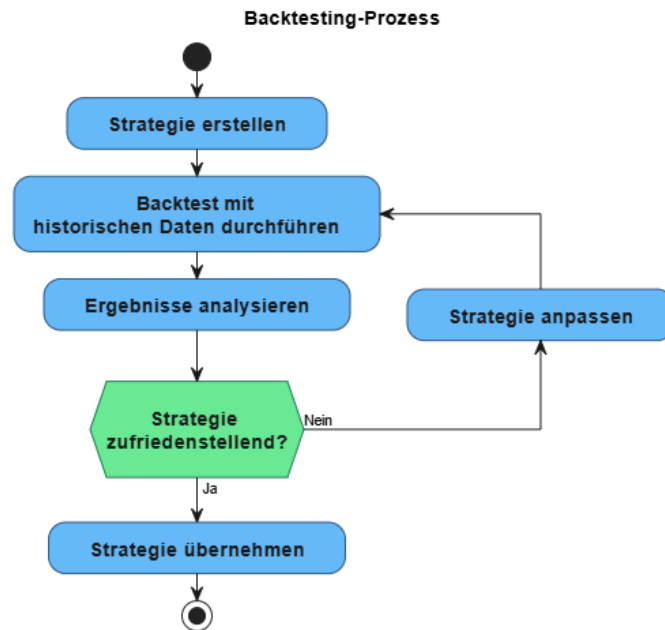


Abbildung 2.1: Ablauf eines Backtestingsprozesses

In der Abbildung 2.1 ist der Ablauf eines Backtestingprozesses dargestellt. Zuerst wird eine Investmentstrategie erstellt, die getestet werden soll. Wie die Strategien implementiert sein muss hängt von dem verwendeten Backtester ab. Daraufhin wird die Strategie auf den historischen Daten im Backtester angewendet. Die Ergebnisse dieses Tests müssen anschließend analysiert werden, um die Qualität der Strategie bewerten zu können. Dazu können verschiedenste Kennzahlen verwendet werden, wie beispielsweise der prozentuelle Gesamtgewinn oder der Sharpe-Ratio, welcher das Verhältnis von Rendite zu Risiko misst.

Der Sharpe-Ratio wird wie folgt berechnet:

$$\text{Sharpe-Ratio} = \frac{R_p - R_f}{\sigma_p}$$

- R_p = Rendite des Portfolios
- R_f = Risikofreier Zinssatz
- σ_p = Standardabweichung der Überschussrendite des Portfolios

Falls die Ergebnisse den Erwartungen entsprechen, ist der Backtestingprozess abgeschlossen. Andernfalls sollte die Strategie angepasst werden und der Backtest erneut ausgeführt werden. Dabei sollte der Prozess jedoch nicht zu oft wiederholt werden, da sonst die Gefahr besteht, dass die Strategie zu stark an die historischen Daten angepasst wird und unter realen Bedingungen nicht mehr den Erwartungen entspricht. [25] [4] Nur weil eine Strategie erfolgreich im Backtesting war, heißt dies aber nicht, dass diese auch in der Zukunft erfolgreich sein wird. Ein möglicher Grund hierfür könnten Verzerrungen (Biase) in der Strategie oder bei den Testdaten sein, die während des Backtests

einbezogen wurden. Das Ergebnis erscheint dadurch im Test überzeugend, in der Praxis aber wird die Strategie den Erwartungen voraussichtlich nicht gerecht. [5]

Desweiteren kann im Backtester ein verfälschtes Ergebnis entstehen, wenn während des Testprozesses keine Gebühren oder Steuern berücksichtigt werden, die in der Praxis anfallen würden. Strategien mit einer geringen Anzahl an Transaktionen wirken im Backtest weniger profitabel als Strategien mit hoher Handelsfrequenz. In der Realität können sie jedoch aufgrund niedrigerer Transaktionskosten und geringerer Slippage oft bessere Ergebnisse erzielen. [9]

2.2 Biase in Backtesting

Durch falsches Anwenden eines Backtesters können verschiedene Biase in die Ergebnisse mit einfließen und diese dadurch verfälschen. Nachfolgend werden vier bekannte Biase beschrieben und wie diese vermieden werden können.

2.2.1 Look-Ahead Bias

Investitionsstrategien, die Informationen verwenden, die sie zum Zeitpunkt der Entscheidung noch nicht haben konnten, sind vom Look-Ahead Bias betroffen. Dieser verfälscht das Backtestergebnis. Wenn die Strategie die Aktienkurse der folgenden Jahre in die Entscheidungsfindung mit einfließen lässt, kann diese Aktien, bei denen ein Kursanstieg erwartet wird, kaufen und Aktien, bei denen ein Kursrückgang erwartet wird, verkaufen. Somit wird die Strategie im Backtest einen Gewinn erzielen, aber bei realen Bedingungen, bei denen nicht in die Zukunft geblickt werden kann, versagen. Um dies zu verhindern, sollten ausschließlich Informationen berücksichtigt werden, die zum Zeitpunkt der Entscheidung tatsächlich vorlagen. [18]

2.2.2 Overfitting Bias

Der Overfitting Bias entsteht, wenn die erstellte Handlungsstrategie zu stark an die ausgewählten historischen Daten angepasst wurde, um bessere Ergebnisse zu erzielen. Dadurch wird die Strategie nur auf diese speziellen Datensets gute Ergebnisse liefern, in der Praxis aber wird dadurch voraussichtlich die Erwartung nicht erfüllt. Ein Beispiel für eine angepasste Strategie sind statisch definierte Kauf- und Verkaufsgrenzen. Um dies zu vermeiden, sollte die Strategie auf mehreren verschiedenen Datensets getestet werden. Ebenfalls sollte zur Erstellung der Strategie ein anderer Teil der historischen Daten verwendet werden als zum Testen der Strategie. [6]

2.2.3 Survivorship Bias

Der Survivorship Bias entsteht, wenn in den historischen Daten nur Firmen enthalten sind, die während des beobachteten Zeitraums weiterhin bestanden haben. Firmen, die in der Vergangenheit in Konkurs gegangen sind oder aus anderen Gründen nicht mehr existieren, sind in den Daten deswegen nicht enthalten. Somit wird die Strategie im Backtest nur auf Firmen angewandt, die es geschafft, haben zu überleben und dabei wird nicht getestet, ob die Strategie auch bei Firmen erfolgreich wäre die in der Vergangenheit gescheitert sind. Um Verzerrungen zu vermeiden, sollten historische Daten einbezogen

werden, die auch Unternehmen berücksichtigen, die im betrachteten Zeitraum insolvent geworden sind. [13] [14]

2.2.4 Data-Snooping Bias

Ein Testergebnis fällt in den Data-Snooping Bias, wenn für einen Test eine zu kleine Auswahl an unterschiedlichen Kursentwicklungen in den Testdatensets verwendet wurde. Daraus entsteht das Problem, dass die Strategie nur auf die in den Datensets vorkommenden Kursentwicklungen optimiert wurde, ohne auf andere Kursentwicklungen Rücksicht zu nehmen. Wenn alle verwendeten Kurse beispielsweise nur von Firmen stammen, die alle in dem betrachteten Zeitraum einen wirtschaftlichen Aufschwung hatten, wird die Strategie auch nur in ähnlichen Szenarien gute Ergebnisse liefern. Somit wird die Strategie in anderen Marktsituationen möglicherweise nicht die Erwartungen erfüllen. Um dies zu vermeiden, sollten Datensätze gewählt werden, die unterschiedliche Aufschwünge und Krisen aufzeigen, um verschiedene Marktsituationen abzudecken. [47] [37]

2.3 State of the Art Backtester

Es gibt bereits zahlreiche Backtester von den verschiedensten Anbietern. Ein Großteil dieser Systeme ist jedoch kostenpflichtig und laufen nicht lokal, sondern nur auf den Servern der einzelnen Anbieter. [66] [65] [56] Somit werden auch selbst entwickelte Strategien an diese Dienstleister weitergegeben. Dritte könnten potenziell die zugrunde liegenden Konzepte der Ansätze übernehmen und für eigene Zwecke adaptieren. Dies ist ein Sicherheitsrisiko für nicht veröffentlichte Strategien. Darüber hinaus ist bei solchen Anbietern häufig auch nicht transparent, wie die Backtester intern implementiert sind. Im Rahmen dieser Arbeit werden daher nur lokale Open-Source Backtester analysiert. Dadurch lassen sich die genannten Probleme vermeiden und die Backtester im Detail untersuchen und besser miteinander vergleichen. Nachfolgend werden drei bekannte open-source Backtester beschrieben und deren Aufbau und Funktionsweise erläutert.

2.3.1 Backtrader

Backtrader ist ein in Python implementierter Backtester. Alle Komponenten des Backtesters sind in einzelne Klassen aufgeteilt. Eine Klasse für Finanzdaten, Backtesteinstellungen und Startparameter, eine für die Strategie, eine für einen Kauf oder Verkauf und eine Klasse für alle errechneten Ergebnisse. Um selbst Strategien zu implementieren, muss eine Strategieklassse erstellt werden. Die eigene Strategieklassse muss von einer Basisklassse erben und gewisse Funktionalitäten implementieren. Von der Art der Strategie hängt ab, ob alle oder nur ein Teil der Funktionen implementiert werden muss. Mit geerbten Funktionen können beispielsweise Kauf- und Verkaufssignale an den Backtester gesendet werden.

Backtrader bietet keine graphische Benutzeroberfläche, um Backtests zu starten, durchzuführen oder zu analysieren. Der Backtest wird durchgeführt, indem die selbst erstellte Python-Datei ausgeführt wird. Als Datenquelle können CSV-Dateien oder Daten von Online-Quellen wie Yahoo Finance verwendet werden. Alle diese Quellen müssen in ei-

nem bestimmen Format vorliegen. Die Resultate des Backtests werden von Backtrader nicht automatisch visualisiert. Stattdessen müssen die Kennzahlen selbstständig aufbereitet und dargestellt werden. Dadurch gestaltet sich die Präsentation aufwendiger, jedoch wird diese dadurch individuell anpassbarer. Zur Veranschaulichung steht eine Hilfsklasse zur Verfügung, die die Darstellung erleichtert und sich zudem durch Ableitung mit eigenen Grafiken erweitern lässt. Ebenfalls vereinfacht die Bibliothek PyFolio die Visualisierung der Ergebnisse. Mit dieser werden aus den Resultaten verschiedene Diagramme und Tabellen mit Kennzahlen erstellt. [64] [41] [60]

Die Performanz von Backtrader ist durch die Geschwindigkeit von Python begrenzt. Backtrader bietet keine Möglichkeit die langsamsten Teile der Strategie in einer anderen Form zu implementieren, um die Ausführungsgeschwindigkeit zu beschleunigen.

2.3.2 Zipline

Zipline ist ein in Python implementierter Backtester. Dieser wurde ursprünglich von Quantopian entwickelt und wird jetzt von der Community weitergeführt, da 2020 Quantopian den Betrieb eingestellt hat. Die weiterentwickelten Versionen von Zipline haben das Ziel, sich von den ursprünglichen Services von Quantopian zu lösen und die Möglichkeit zu bieten, den Backtester lokal auszuführen. Zwei der bekanntesten Forks sind Zipline Trader und Zipline Reloaded. Den Entwicklern ist es gelungen, sich von den eingestellten Services zu lösen und eigene Funktionalität weiter hinzuzufügen. Beide Forks sind Open-Source und können kostenlos verwendet werden. Ebenfalls ist es möglich beide Systeme über die Kommandozeile zu bedienen, mit Python-Skripten zu steuern oder diese in einem Jupiter Notebook anzuwenden. Zipline Trader geht mehr in die Richtung von Python Skripten, während Zipline Reloaded mehr für die Verwendung in einem Jupiter Notebook oder der Kommandozeile ausgelegt ist.

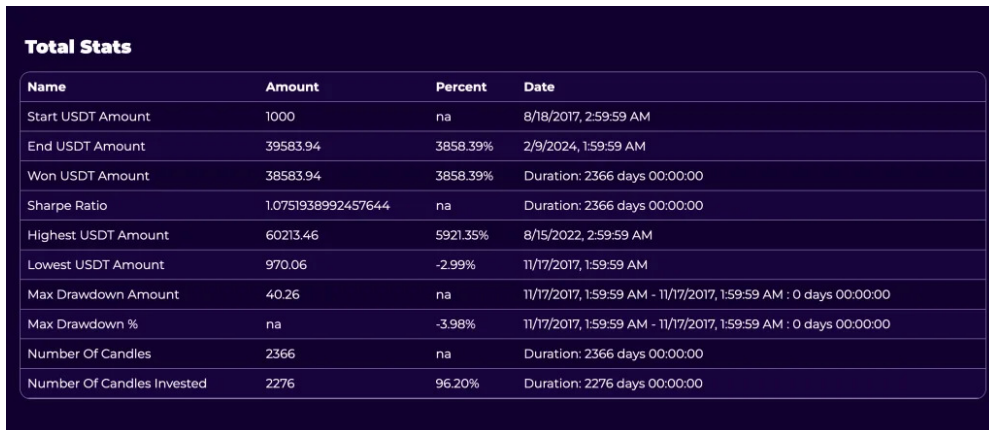
Damit eigene Strategien verwendet werden können muss eine Funktion implementiert werden, die eine Sammlung an Daten nimmt und Kauf- und Verkaufssignale zurückgibt. Dabei können verschiedenste Hilfsfunktionen und Indikatoren verwendet werden, die in Zipline bereits implementiert sind. Zipline Trader benötigt, um historische Daten zu laden eine Verbindung zu Alpaca oder Interactive Brokers und eine Postgres-Datenbank. Zipline Reloaded hat standardmäßig Kurse für gewisse Aktien vorinstalliert und kann auch Daten von NASDAQ im pickle Format über die API nachladen.

Die Ergebnisse eines Backtests werden in beiden Systemen automatisch generiert aber müssen manuell visualisiert werden. Dafür können verschiedene Bibliotheken wie Matplotlib, PyFolio oder Quantstats verwendet werden. [61] [31] [63] [62] [32]

2.3.3 BacktestJS

BacktestJS ist ein in TypeScript und JavaScript geschriebener Backtester. Dieser kann nur aus der Kommandozeile heraus verwendet werden und hat keine grafische Benutzeroberfläche. Die entstandenen Ergebnisse liegen als HTML-Dateien vor und können in einem Webbrowser betrachtet werden. In diesem Backtester ist es auch möglich eigene Strategien testen zu lassen. Dazu muss eine Strategie in JavaScript oder TypeScript implementiert werden die gewisse Bedingungen erfüllt. Diese Regeln bestimmen wie die Funktion zu heißen hat, welche Parameter diese annimmt, welche Imports nötig sind und wo diese sich im Dateisystem zu befinden hat. Ebenfalls stellt BacktestJS die Mög-

lichkeit zur Verfügung sich historische Finanzdaten in eine lokale Datenbank zu laden und diese als CSV-Datei zu exportieren. Diese Daten stammen von Binance und sind kostenlos verfügbar. Andere CSV-Dateien in diesem Format können auch direkt in die Datenbank mit BacktestJS geladen werden. Damit ist es möglich seine Strategien auf verschiedene Kurse zu testen. [57] [8]



Total Stats			
Name	Amount	Percent	Date
Start USDT Amount	1000	na	8/18/2017, 2:59:59 AM
End USDT Amount	39583.94	3858.39%	2/9/2024, 1:59:59 AM
Won USDT Amount	38583.94	3858.39%	Duration: 2366 days 00:00:00
Sharpe Ratio	1.0751938992457644	na	Duration: 2366 days 00:00:00
Highest USDT Amount	60213.46	5921.35%	8/15/2022, 2:59:59 AM
Lowest USDT Amount	970.06	-2.99%	11/17/2017, 1:59:59 AM
Max Drawdown Amount	40.26	na	11/17/2017, 1:59:59 AM - 11/17/2017, 1:59:59 AM : 0 days 00:00:00
Max Drawdown %	na	-3.98%	11/17/2017, 1:59:59 AM - 11/17/2017, 1:59:59 AM : 0 days 00:00:00
Number Of Candles	2366	na	Duration: 2366 days 00:00:00
Number Of Candles Invested	2276	96.20%	Duration: 2276 days 00:00:00

Abbildung 2.2: Beispielergebnis eines Backtests mit BacktestJS [7]

Ein Beispiel für ein Ergebnis eines Backtests mit BacktestJS ist in der Abbildung 2.2 zu sehen. In dieser Statistik werden verschiedene Kennzahlen wie beispielsweise der Gesamtgewinn, der Startbetrag, der Höchststand der Aktie und der Sharpe-Ratio angezeigt. Es werden nach einem Backtest auch noch weitere Tabellen und Diagramme mit genaueren Informationen generiert und angezeigt.

Die Geschwindigkeit von BacktestJS ist limitiert durch die Ausführungsgeschwindigkeit von JavaScript. Es ist nicht möglich rechenintensive Teile der Strategie in einer anderen schnelleren Programmiersprache wie C oder C++ zu schreiben, um die Backtest-Prozess zu beschleunigen. Dies schränkt den Rahmen für Optimierungen der Laufzeit bei komplexeren Strategien ein.

2.4 Bewährte Ansätze für Handelsstrategien

Es gibt viele bereits etablierte Ansätze für Anlagestrategien. In dieser Arbeit werden keine neuen Algorithmen entwickelt, sondern bestehende Ansätze verwendet, diese in JavaScript und C implementiert, in einem Webbrowser ausgeführt und die Ausführungsgeschwindigkeit getestet. Nachfolgend werden drei bereits bekannte Ideen für Investitionsstrategien beschrieben.

2.4.1 Mittelwertrückkehr mit Bollinger Bänder

Die Idee hinter der Mittelwertrückkehr ist, dass der Kurs einer Aktie immer wieder zu einem längerfristigen Mittelwert zurückkehrt. Dabei werden Abweichungen von dem Mittelwert als Kauf- oder Verkaufssignal verstanden. Es werden die Aktien verkauft,

wenn der Kurs über dem Mittelwert steigt und gekauft, wenn der Kurs unter dem Mittelwert fällt. [38]

Um die Signale zu verstärken und nicht bei den kleinsten Änderungen eine Aktion auszuführen werden häufig Bollinger Bänder verwendet. Diese bestehen aus drei Komponenten.

- Einem gleitenden Durchschnitt des Kurses über einen bestimmten Zeitraum.
- Einem oberen Band, welches eine bestimmte Anzahl an Standardabweichungen über dem Durchschnitt liegt.
- Einem unteren Band, welches eine bestimmte Anzahl an Standardabweichungen unter dem Durchschnitt liegt.

Somit wird ein breiter Mantel über den Aktienkurs gelegt, der die normalen Schwankungen des Kurses umhüllt. Wenn der Kurs das Band nach oben hin durchbricht, wird dies als Verkaufssignal gewertet. Unterschreiten des Bandes wird als Kaufsignal gewertet. [11] [46] [35]

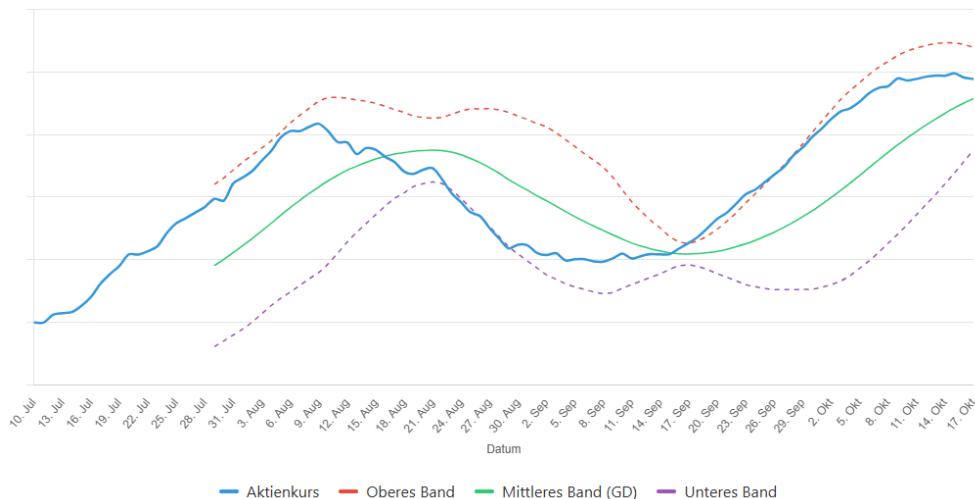


Abbildung 2.3: Beispiel für ein Bollinger Band

In der Abbildung 2.3 ist ein Beispiel für ein Bollinger Band zu sehen. Der blaue Bereich stellt den Kursverlauf der Aktie dar. Die grüne Linie ist der gleitende Durchschnitt, das obere Band ist die rote Linie und das untere Band ist die violette Linie. Am 17. September ist ein Verkaufssignal zu erkennen, da der Kurs das obere Band durchbricht. Am 21. August ist ein Kaufsignal zu erkennen, da der Kurs das untere Band unterschreitet.

Wechselkurse von Währungen dienen als Beispiel dafür, dass Kurse oft zu einem Mittelwert zurückkehren. Diese pendeln häufig um einen Mittelwert und bleiben im Verhältnis zueinander stabil. [21]

Bei Strategien mit Mittelwertrückkehr kann es zu Problemen kommen, wenn der Kurs nicht zum Mittelwert zurückkehrt, sondern sich stark in eine Richtung bewegt. Beispiele dafür sind Firmen, die in Konkurs gehen oder einen starken wirtschaftlichen Aufschwung erleben. [40]

2.4.2 Gleitende Durchschnitts Überschneidungen

Gleitende Durchschnitts Überschneidungen Strategien sind eine der bekanntesten und am häufigsten verwendeten Ansätze im Bereich des automatisierten Handels. Dabei werden zwei gleitende Durchschnitte mit unterschiedlichen langen Zeiträumen errechnet und miteinander verglichen. Wenn der kurzfristige Durchschnitt den langfristigen Durchschnitt von unten nach oben durchbricht, wird dies als Kaufsignal gewertet. Wird der langfristige Durchschnitt aber von oben nach unten durchbrochen, wird dies als Verkaufssignal gewertet. Der Unterschied in den verschiedenen Strategien mit dieser Vorgehensweise liegt in der Wahl der Zeiträume für die beiden gleitenden Durchschnitte und der Mehrgewichtung aktueller Werte. [29] [12]

Eine optimierte gleitende Durchschnitts Überschneidungsstrategie erzielt mehr Rendite als die Strategie eine Aktie zu kaufen und zu warten. Für den S&P 500 Index zeigte die wartende Strategie im Zeitraum von März 2009 bis November 2014 einen niedrigeren Sharpe-Ratio (1,214) als die optimierte gleitende Durchschnittsüberschneidungsstrategie (1,351). Dies bedeutet, dass die optimierte Strategie im Verhältnis zu dem eingegangenen Risiko eine höhere Rendite erzielen konnte. [23]

Ein Problem bei gleitenden Durchschnitts Überschneidungsstrategien ist, dass es zu Fehlsignalen kommen kann, wenn die beiden Durchschnitte sich häufig überschneiden. Dies passiert häufig in Seitwärtsmärkten, bei denen der Kurs keine klare Richtung hat und sich nur wenig verändert. Dabei kann es dazu kommen, dass es sehr viele Kauf- und Verkaufssignale gibt, die einerseits zu hohen Transaktionskosten führen und andererseits bei langsam fallenden Kursen zu Verlusten führen. [3]

Ein weiteres Problem ist, dass diese Strategien sehr von der Auswahl der Zeiträume für die gleitenden Durchschnitte abhängen. Durch eine schlechte Wahl der Zeiträume kann es dazu kommen, dass die Strategie nicht die Erwartungen erfüllt. [20]

2.4.3 Ausbruchsstrategien

Bei Ausbruchsstrategien wird davon ausgegangen, dass der Aktienkurs aus gesetzten Grenzen ausbricht und sich in die Richtung des Ausbruchs weiterbewegt. Zur Bestimmung der Grenzen werden häufig Hoch- und Tiefpunkte in der Kursentwicklung in einem gewissen Zeitrahmen verwendet. Ausbrüche über die Obergrenze werden als Kaufsignal gewertet, während Ausbrüche unter die Untergrenze als Verkaufssignal gewertet werden. [27] [1]

Mit einer optimierten Ausbruchsstrategie kann sogar der S&P 500 Index in einem Backtest übertroffen werden. Im Zeitraum vom Jänner 2016 bis Dezember 2023 konnte der S&P 500 198% Rendite erzielen, während eine optimierte Ausbruchsstrategie 1637% Rendite erzielen konnte. Die Strategie war Risikoreicher als der S&P 500, konnte aber durch das höhere Risiko auch eine deutlich höhere Rendite erzielen. [53]

Ein Problem bei Ausbruchsstrategien ist, dass es zu Fehlsignalen kommen kann. Dies passiert, wenn eine der Grenzen kurzfristig durchbrochen wird, ohne weiter in die durchbrochene Richtung zu laufen. Durch diese Fehlsignale kann es dazu kommen das bei Spitzen gekauft und in Tälern verkauft wird, was zu Verlusten führt. [39]

2.5 WASM

WebAssembly (WASM) ist ein binäres Instruktionsformat für das Web. Es wurde dafür entwickelt, Webanwendungen schneller und effizienter ausführen zu können und diese dabei gleichzeitig portable zu halten. WebAssembly kann in verschiedensten höheren Programmiersprachen wie C, C++, Rust und Go geschrieben werden und wird von allen modernen Webbrowsern unterstützt. Der erstellte Quellcode kann anschließend mit einem Compiler, beispielsweise Emscripten, in Bytecode übersetzt werden und durch Aufrufen mit JavaScript in einem Browser ausgeführt werden. [22]

```
int add(int a, int b) {  
    return a + b;  
}  
  
(module  
  (type (;0;) (func (param i32 i32) (result i32)))  
  (func (;0;) (type 0) (param i32 i32) (result i32)  
    local.get 1  
    local.get 0  
    i32.add)  
  (export "add" (func 0))  
)
```

Abbildung 2.4: Vergleich von C Quellcode und generiertem WebAssembly Bytecode

In der Abbildung 2.4 ist ein Beispiel für C Quellcode und der daraus generierte WebAssembly Bytecode zu sehen. In C wird eine Funktion definiert, die zwei ganzzahlige Werte als Parameter annimmt, diese addiert und zurückgibt. Im WebAssembly-Bytecode wird in der zweiten Zeile zunächst der Funktionsprototyp erstellt. Dieser erhält zwei Ganzzahlen als Eingabeparameter und eine Ganzzahl als Rückgabewert. Anschließend erfolgt die Definition der Funktion selbst. In der Funktion werden die beiden lokalen Variablen die als Parameter übergeben wurden, geladen und addiert. Das Ergebnis verbleibt auf dem Stack und stellt dadurch den Rückgabewert der Funktion dar. Das Schlüsselwort `export` gibt den Namen der Funktion an, damit diese von außen aufgerufen werden kann. Die Metadaten, die durch den Compiler hinzugefügt werden, sind dabei im Bytecode weggelassen. Ebenfalls sind die Präprozessoranweisungen im C Quellcode, um ihn mit Emscripten kompilieren zu können nicht dargestellt.

Da WebAssembly nicht direkt mit dem Browser interagieren kann, gibt es einige Einschränkungen. So hat WebAssembly beispielsweise keinen direkten Zugriff auf das Document Object Model (DOM) eines Webbrowsers. Ebenfalls gibt es keinen automatischen Garbage Collector, was zu Folge hat, dass die Speicher manuell verwaltet werden muss. [22]

Des Weiteren können Funktionen in WebAssembly nur einzelne Werte und keine Verbunde oder Objekte zurückgeben. Damit ist also weniger Sprachumfang als in JavaScript gegeben. Es kann ebenfalls auch nicht auf Umgebungsvariablen beispielsweise dem DOM zugegriffen werden. [24]

2.5.1 Ressourcenverbrauch und Performanz von WASM

Der Vergleich der Ausführungszeiten zeigt, dass WebAssembly in den von [52] getesteten Desktop-Browsern insgesamt schneller ausgeführt wird als JavaScript. Während Firefox JavaScript im Durchschnitt etwas langsamer ausführt als Chrome ($1,06\times$ Ausführungszeit), läuft WebAssembly in Firefox deutlich schneller und benötigt nur $0,61\times$ der Ausführungszeit von Chrome. Auch im direkten Vergleich erreicht WebAssembly in Firefox kürzere Laufzeiten als reines JavaScript, aufgrund optimierter Funktionsaufrufe und effizienterer Ausführungspfade. Die absoluten Ausführungszeiten zeigen aber, dass WebAssembly nur in Firefox schneller ist als JavaScript (39,65 ms gegenüber 48,26 ms), während es in Chrome und Edge jeweils langsamer ausgeführt wird als JavaScript. Bei der Laufzeit weist WebAssembly im Vergleich zu JavaScript ebenfalls einen vielfach höheren Speicherverbrauch auf. In allen Desktop-Browsern benötigt WebAssembly deutlich mehr Speicher als JavaScript (3,39x in Chrome, 4,93x in Firefox und 3,44x in Edge). [52]

Auch andere Studien zeigen, wie WebAssembly im richtigen Browser schneller ausgeführt wird als JavaScript. In den Tests von [19] kommt WebAssembly mit einem optimiertem Speicherzugriff und parallelem Datenzugriff beim Multiplizieren einer 102×1024 Matrix auf einen 1,64-fachen Geschwindigkeitsvorteil gegenüber einer optimierten JavaScript-Implementierung. [48] zeigt desweiter, dass WebAssembly besonders bei rechenintensiven Aufgaben wie der Bild- und Videobearbeitung oder mathematischen Simulationen eine deutlich bessere Leistung als JavaScript bietet.

Im Vergleich zu nativen Anwenden in C schneidet WebAssembly jedoch schlechter ab. WebAssembly erreicht im Durchschnitt nur etwa 60-70% der Leistung von nativem C-Code. [26] [33]

2.6 Laufzeitumgebungen für WebAssembly

Um WebAssembly im Browser ausführen zu können wird eine geeignete Laufzeitumgebungen benötigt. Dazu werden in den unterschiedlichen Browser verschiedene Umgebungen eingesetzt.

Firefox nutzt zur Ausführung von WebAssembly die Engine SpiderMonkey. SpiderMonkey ist eine von Mozilla entwickelte JavaScript- und WebAssembly-Laufzeitumgebung, die in den Browser interagiert ist. Diese ist überwiegend in C++ implementiert und wird durch Komponenten in JavaScript und Rust ergänzt. [43]

Chrome nutzt die V8 Engine, eine von Google entwickelte JavaScript- und WebAssembly-Laufzeitumgebung. Diese ist in C++ implementiert. V8 wird auch in anderen Chromium-basierten Anwendungen verwendet. [50]

Beide Laufzeitumgebung sind auf Basis der WebAssembly-Spezifikation implementiert und ermöglichen es WebAssembly sicher und plattformunabhängig auszuführen. Unterschiede zwischen den Engines ergeben sich durch die genaue Implementierung der einzelnen Browserhersteller und die Art der Optimierungen.

2.6.1 Ausführung der WebAssembly-Anweisungen

Die Übersetzung von WebAssembly-Bytecode in ausführbare, browserspezifische Instruktionen erfolgt in beiden Laufzeitumgebungen strukturell gleich.

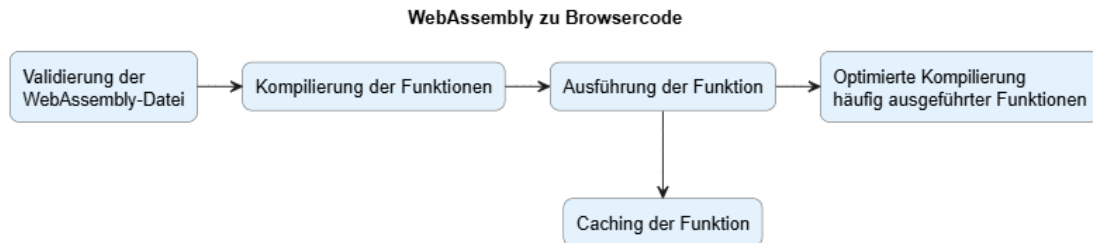


Abbildung 2.5: Ausführung von WebAssembly im Browser

In der Abbildung 2.5 ist dargestellt wie WebAssembly-Code in Instruktionen für einen spezifischen Browser umgewandelt wird. Zuerst wird die WebAssembly-Datei überprüft, ob diese in einem validen Format laut WebAssembly-Spezifikation vorliegt. Danach werden die gefundenen Funktionen in browserspezifischen Code kompiliert. SpiderMonkey generiert direkt beim Interagieren der WebAssembly-Datei für jede Funktion den Code. Die V8 Engine hingegen erzeugt den Code für die Funktionen erst wenn diese zum ersten Mal aufgerufen werden. Die Ausführung der Funktion erfolgt daraufhin basierend auf dem neu erstellten Code. Wird eine Funktion häufig aufgerufen, wird diese erneut mit stärker optimiertem Code kompiliert. Jede der Funktionen kann nach der Ausführung gecacht werden, damit diese nicht bei jedem weiteren Aufruf erneut kompiliert werden muss. [49] [44] [24] [22]

Kapitel 3

Konzept und Design

In diesem Kapitel werden das Konzept und Design des im Rahmen dieser Arbeit entwickelten Backtesters beschrieben. Zudem wird der Aufbau der Strategien sowie die Herangehensweise bei der Visualisierung der Kennzahlen und Ergebnisse erläutert.

3.1 Architektur des Backtesters

Der entwickelte Backtester besteht aus mehreren Komponenten. Den erhaltenen Daten, der Backtesterlogik und der Visualisierung der Ergebnisse.

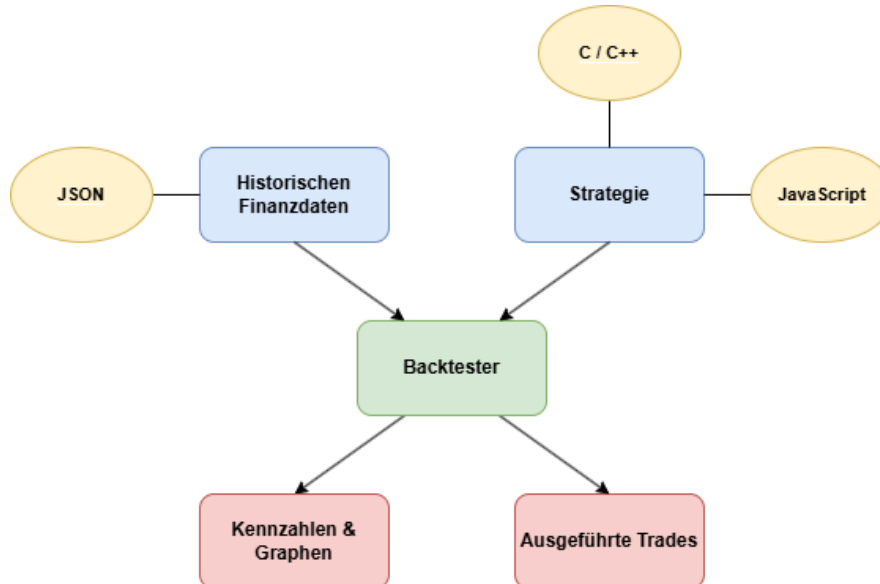


Abbildung 3.1: Struktur des Backtesters

In der Grafik 3.1 ist die Struktur des gesamten Backtesters dargestellt. Zu Beginn werden eingehenden Daten in die Backtesterlogik eingespeist. Einerseits wird eine Historie von Kursdaten im JSON Format übergeben, andererseits wird die zu testende Strategie als WebAssembly Modul oder JavaScript Datei geladen. Als Result entsteht eine

Visualisierung der Backtestergebnisse und die Auflistung der Trades.

3.1.1 Format der historischen Finanzdaten

Die historischen Finanzdaten werden über die Weboberfläche im JSON Format an die Backtesterlogik weitergereicht. Das JSON Format eignet sich für die Übergabe, da es von JavaScript nativ unterstützt wird und leicht zu parsen ist. JSON hat sich als Standardformat für die Datenverwaltung im Web etabliert, da dieses gut Menschen-lesbar ist und performant von Webbrowsern verarbeitet werden kann. Darüber hinaus bringt die Struktur von JSON den Vorteil mit sich, dass komplexe Datenstrukturen damit einfach abgebildet werden können. XML wurde nicht gewählt, da es im Vergleich zu JSON eine umfangreichere Syntax besitzt und dadurch mehr Speicherplatz sowie Rechenzeit beim Parsen benötigt. Des Weiteren ist XML weniger kompakt und wird von modernen Webanwendungen nicht so effizient wie JSON verarbeitet. [54]

Die historischen Daten müssen in einer bestimmten Form vorliegen, um eine korrekte Verarbeitung durch die Backtesterlogik zu ermöglichen. Jeder Datenpunkt ist dabei durch zwei Attribute zu beschreiben: einen Zeitstempel im ISO Format sowie einen zugehörigen Kurswert als Flieskommazahl.

3.1.2 Backtesterlogik

In der Backtesterlogik ist der Hauptteil der Funktionalität des Backtesters umgesetzt. Der Backtestprozess beginnt, nachdem die historischen Finanzdaten und die zu testende Strategie geladen wurden. Dabei wird iterativ über den gesamten Datensatz gegangen und in einem vordefinierten Intervall die Strategie aufgerufen und nach einem Handelssignal befragt. Die Strategie erhält als Eingabeparameter den aktuellen Kurswert, eine festgelegte Anzahl historischer Datenpunkte sowie den zeitlichen Abstand, in dem diese historischen Kurswerte voneinander entfernt erfasst wurden. Die WASM Module erhalten ebenso die Anzahl der übergebenen Kurswerte, damit die Daten korrekt verarbeitet werden können. Je nach zurückgegebenem Signal der Strategie wird danach ein Kauf- oder Verkaufsvorgang simuliert. Dieser Vorgang wird aber nur durchgeführt, wenn bei einem Kaufvorgang genügend Kapital vorhanden ist oder bei einem Verkaufsvorgang genügend Wertpapiere im Portfolio sind. Somit wird verhindert, dass ein negatives Kapital oder ein negativer Bestand an Wertpapieren entsteht. Jede erfolgreiche Transaktion während des Backtestprozesses wird protokolliert, um diese nach vollständiger Berechnung visualisieren zu können. In der Abbildung 3.2 ist dieser Prozess visuell dargestellt.

Ablauf der Backtester Logik**Abbildung 3.2:** Backtestingablauf

Ein weiterer Aspekt in der Backtesterlogik ist die Messung der Ausführungsgeschwindigkeit der einzelnen Strategien. Da gleichwertige Strategien mit identer Logik in verschiedenen Programmiersprachen implementiert und getestet werden können, ist es möglich die Ausführungszeit der Strategien zu vergleichen. Dazu wird die Zeit gemessen, die für das Ausführen der Strategie während des Backtestes benötigt wird. Die Zeit startet nach dem Initialisieren des Backtesters und endet, nachdem alle Datenpunkte in den Kursdaten berücksichtigt wurden. Das Laden der historischen Finanzdaten, der Strategie und die Errechnung der Kennzahlen wird dabei nicht mit einbezogen.

Am Ende des Backtestingvorganges werden ebenfalls verschiedene Kennzahlen errechnet, um die Performance der getesteten Strategie zu bewerten. Zu den errechneten Kennzahlen gehören: Endwert des Portfolios, Gesamttrendite, prozentueller Gewinn und der Sharpe Ratio.

3.1.3 Modularisierung der Strategien

Der Backtester ist so gestaltet, dass verschiedene Handelsstrategien modular hochgeladen und getestet werden können. Die Modularisierung ermöglicht es, eigene Strategien zu implementieren und diese in dem Backtester zu verwenden ohne Backtestinglogik verändern zu müssen. Die Strategiemodule müssen dabei nur einen definierten Namen und eine vordefinierte Signatur besitzen, um korrekt aufgerufen werden zu können. Der

Rückgabewert dieser Funktion bestimmt dabei, ob ein Kauf-, Verkaufs- oder Haltesignal ausgegeben wird.

```
int strategy(double *prices, int length, int intervalsBack)

function strategy(prices, intervalsBack)
```

Abbildung 3.3: Funktionsschnittstellen der Strategien in C und JavaScript

In der Abbildung 3.3 sind die Funktionsschnittstellen der Strategien in C und JavaScript dargestellt. Beide Schnittstellen besitzen denselben Funktionsnamen und die logisch gleichen Parameter. Die C-Funktion erhält im Vergleich zur JavaScript Funktion zusätzlich die Anzahl der übergebenen historischen Kurswerte als Parameter, da in C keine dynamischen Arrays unterstützt werden. Beide Funktionen geben einen ganzzahligen Wert zurück, der das Handelssignal repräsentiert.

Das modulare Konzept mit Funktionen erlaubt die Verwendung von Strategien in verschiedenen Programmiersprachen. Der in dieser Arbeit entwickelte Backtester unterstützt Strategien in den Programmiersprachen JavaScript und WebAssembly (WASM), um den Vergleich der Ausführungsgeschwindigkeiten zu ermöglichen. Die Trennung zwischen dem Backtester und den Strategien ermöglicht es das System zukünftig einfach, um weitere Sprachen und Schnittstellen zu erweitern.

Um möglichst viele Programmiersprachen unterstützten zu können, wird der Aufruf der Strategien über eine einfache Funktionsschnittstelle realisiert. Da WebAssembly nur Funktionsaufrufe erlaubt, eignet sich dieser Ansatz besonders gut, um eine breite Kompatibilität zu ermöglichen. In WebAssembly können keine Klassen nach außen definiert werden, weshalb die Funktionsschnittstelle die einzige Möglichkeit darstellt, mit dem Backtester zu kommunizieren. [22] [24]

Desweiteren bietet die Funktionsschnittstelle den Vorteil, dass keine komplexen Objekte oder Datenstrukturen wie Klassen erzeugt werden müssen. Der Aufruf über eine Funktion ist im Vergleich zu objektorientierten Ansätzen in der Regel effizienter, vor allem bei Sprachenübergreifenden Schnittstellen. Dies vereinfacht die Implementierung der Strategien in verschiedenen Sprachen, verringert die Ausführungszeit und reduziert den Aufwand für die Integration in den Backtester. [30] [24]

3.2 Aufbau der Teststrategien

Zum Testen des Backtesters wurden verschiedene Konzepte für Handelsstrategien in konkrete Implementierungen überführt. Jeder der Strategieansätze verfolgt einen anderen Ansatz zur Generierung von Kauf- und Verkaufssignalen und weist dementsprechend auch unterschiedliche Geschwindigkeiten in der Ausführung auf. Der Aufbau der drei umgesetzten Strategien ist in den folgenden Unterkapiteln beschrieben.

3.2.1 Struktur der Mittelwertrückkehr Strategie mit Bollinger Bändern

Die Durchschnittsrückkehr beschreibt die Tendenz von Kurswerten, nach Abweichungen wieder in die Nähe ihres Mittelwerts zurückzukehren. Die Verwendung von Bollinger Bändern reduziert die Häufigkeit schwacher Handelssignale, indem kurzfristige Schwankungen herausfiltert werden und nur starke Preisabweichungen Impulse auslösen. Eine detaillierte Beschreibung dieser Strategieidee befindet sich in Abschnitt 2.4.1.

Als Breite des Bollinger Bandes wird eine Standardabweichung von 3,25 gewählt, um die Strategie an längerfristige Anlagen und Märkte mit höherer Volatilität anzupassen. Laut [34] sind Breiten von 1,8 Standardabweichungen von Bändern bei kurzzeitigen Investments vorteilhaft. Je mehr Schwankungen im Kursverlauf enthalten sind und bei einem längeren Anlagehorizont sollte eine größere Standardabweichung gewählt werden, um die Breite des Bandes entsprechend anzupassen. Somit werden mehr kurzfristige Signale entfernt und das Risiko für Fehlsignale gesenkt. Die Strategie löst erst dann Trades aus, wenn genügend Datenpunkte übergeben werden, da für die Berechnung der Standardabweichung eine Reihe von Werten benötigt wird. Die Mindestanzahl an historischen Kurswerten wird in der Strategie selbst festgelegt.

3.2.2 Aufbau der Ausbruchsstrategie

Ausbruchsstrategien erzeugen ein Kauf- oder Verkaufssignal, wenn der aktuelle Kurswert das in einem beobachteten Intervall ermittelte Maximum überschreitet oder das Minimum unterschreitet. Eine genauere Erklärung dazu befindet sich in dem Kapitel 2.4.3.

Die für diese Arbeit entwickelte Ausbruchsstrategie bezieht in die Minima und Maximum Berechnung alle erhaltenen Wert bis auf den Letzten mit ein. Der letzte Kurswert wird jedoch nicht direkt mit den Extremwerten des vergangenen Beobachtungszeitraums verglichen, sondern um geringfügige Kursschwankungen, die keinen tatsächlichen Ausbruchstrend anzeigen, zu vermeiden, mit einem Epsilon-Faktor angepasst. Dadurch sollen Fehlsignale in Seitwärtsmarktphasen vermieden werden. Ebenfalls zur Absicherung gegen Fehlsignale werden Haltesignale erst dann ausgegeben, wenn der Strategie mindestens die eingegebene Anzahl an Kurswerten übergeben wurde.

3.2.3 Umsetzung der gleitenden Durchschnittsüberschneidungs Strategie

Bei der Durchschnittsüberschneidungsstrategie werden zwei gleitende Durchschnitte mit unterschiedlich langen Zeiträumen berechnet und miteinander verglichen, um Handelssignale zu erzeugen. Wie bereits in Kapitel 2.4.2 beschrieben.

Die implementierte Handelsstrategie verwendet dabei einen gleitenden Durchschnitt mit einer Länge von acht Zeiteinheiten als kurzfristigen Durchschnitt, während für den langfristigen Durchschnitt ein Zeitraum von 24 Zeiteinheiten beobachtet wird. Dieses 1:3 Verhältnis wurde so gewählt, da es eine gute Balance zwischen Reaktionsfähigkeit, Gewinn und Stabilität bietet. [12] Diese Strategie berücksichtigt nur die aktuellsten 24 Datenpunkte, um die Durchschnitte zu berechnen. Alle übergebenen historischen Datenpunkte, die älter als dieses Intervall sind, werden ignoriert. Die Strategie generiert ausschließlich Haltesignale, sobald eine ausreichende Anzahl an Datenpunkten zur Berechnung der gleitenden Durchschnitte vorliegt. Zudem werden Handelssignale nur dann

ausgelöst, wenn die Überschneidung der Durchschnitte einen definierten prozentualen Schwellenwert in Relation zum aktuellen Kurs überschreitet, um die Entstehung von Fehlsignalen zu reduzieren.

3.3 Weboberfläche des Backtesters

Die Weboberfläche des Backtesters ermöglicht es, historische Kursdaten und Strategien hochzuladen, Backtests zu starten und die Ergebnisse zu visualisieren. Der Backtester ist nur über die Weboberfläche bedienbar. Die Webseite ist in verschiedene Bereiche unterteilt, die jeweils unterschiedliche Funktionen bereitstellen.

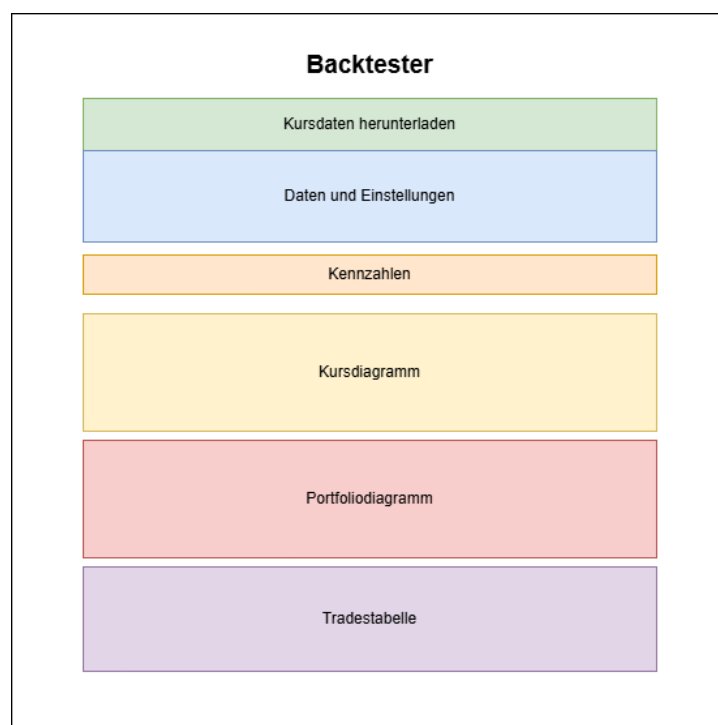


Abbildung 3.4: Design der Weboberfläche des Backtesters

In der Abbildung 3.4 ist das Design der Weboberfläche des Backtesters dargestellt. Der oberste Bereich bietet die Möglichkeit, historische Kursdaten im JSON Format herunterzuladen. Dabei muss das Kürzel des gewünschten Wertpapiers sowie der Zeitraum angegeben werden. Unterhalb befindet sich der Abschnitt, in dem die Kursdaten und die Handelsstrategie hochgeladen sowie die Startparameter für den Backtest festgelegt werden können. Zu diesen Parametern zählen das Anfangskapital, das Intervall, in dem die Strategie abgefragt werden soll, sowie die Anzahl der historischen Datenpunkte, die an die Strategie übergeben werden. Nachdem alle Felder ausgefüllt und als gültig erkannt wurden, kann der Backtest per Knopfdruck gestartet werden. Im folgenden Bereich werden nach der Berechnung die Kennzahlen des Backtests dargestellt. Daraufhin werden die Kursverläufe mit den Handelszeitpunkten sowie die Portfolioentwicklung visualisiert. Abschließend wird eine Tabelle mit allen im Rahmen des Backtests ausgeführten

Trades angezeigt.

3.4 Visualisierung der Daten und Ergebnisse

Die Kennzahlen, Ergebnisse des Backtestes und Kursdaten werden als Liniendiagramme visualisiert. Dabei werden diese in zwei Diagrammen dargestellt. 3.5 Das obere Diagramm zeigt die Kursdaten über die Zeit an und markiert an welchen Stellen ver- oder gekauft wurde. Das untere Diagramm stellt den Portfoliowert über die Zeit dar. Das Portfoliodiagramm und die Kennzahlen helfen die Effektivität der getesteten Strategie zu einzuschätzen und machen die Performance visuell sichtbar.

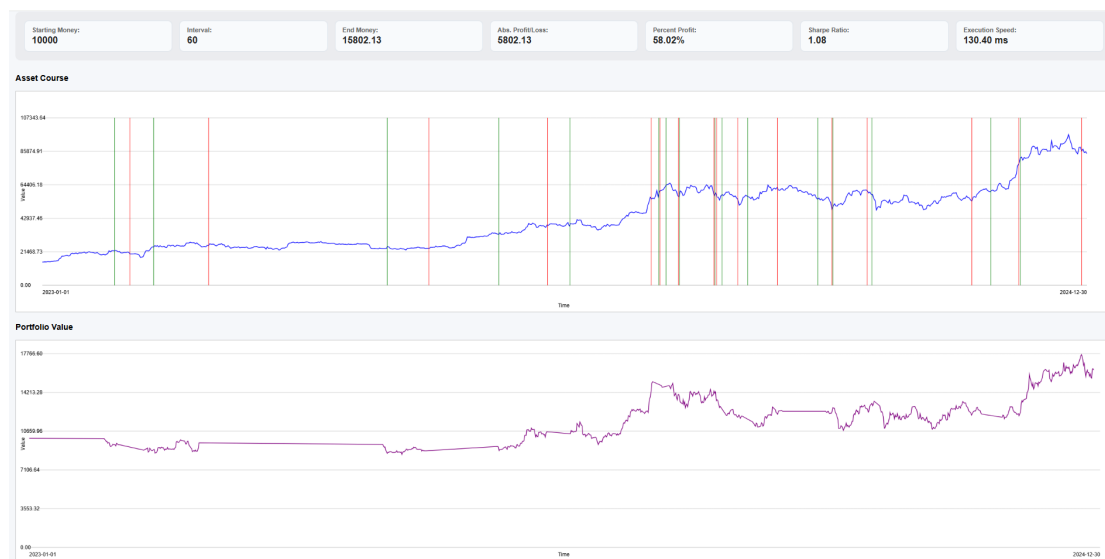


Abbildung 3.5: Kursdiagramme mit Kennzahlen

Bei der Visualisierung der Finanzdaten fallen große Datenmengen mit Millionen von Datenpunkten an. Um eine übersichtliche Darstellung zu ermöglichen, werden die Datensätze daher zur Darstellung durch Downsampling reduziert. Zur Reduktion des Datensets wird eine stichprobenartige Reduktion mittels Schrittweite verwendet, bei der jeder n-te Datenpunkt berücksichtigt wird. Dadurch wird die Anzahl der dargestellten Datenpunkte verringert, während die grundlegende Form des Aktienkurses erhalten bleibt. Durch die Wahl der Schrittweite kann die Detailliertheit der Darstellung angepasst werden. [36]

3.5 Aufbau der Geschwindigkeitstests

Zur Evaluierung der Ausführungsgeschwindigkeit der implementierten Strategien in den verschiedenen Programmiersprachen wird bei jedem Backtest die Laufzeit erfasst. Da der Fokus auf dem Vergleich der Ausführungszeiten zwischen den Programmiersprachen liegt, wird die Zeitmessung bei mehreren identen Strategielogiken in verschiedenen Sprachen durchgeführt. Als Ausführungsumgebungen werden die Webbrowser Google

Chrome und FireFox verwendet. Google Chrome ist mit etwa 70 % Marktanteil, der am häufigsten genutzte Browser und bietet somit eine realistische Einschätzung der Performance. [45, 51] FireFox wird als zweiter Testbrowser eingesetzt, da dieser die schnellste Ausführungsgeschwindigkeit für WebAssembly Module bietet und somit einen interessanten Vergleichspunkt darstellt, wie in Kapitel 2.5.1 beschrieben.

Die Backtests werden mit unterschiedlich großen Datensätzen durchgeführt, um die Performance der Strategien bei variierenden Datenmengen zu bewerten. Ebenfalls werden alle Tests auf derselben Hardware ausgeführt, um eine Vergleichbarkeit der Ergebnisse zu gewährleisten. Zusätzlich wird jede Strategie mehrfach auf demselben Datensatz angewendet, um die gemessenen Laufzeiten zu Mitteln und zufällige Ausreißer durch Hintergrundprozesse oder Just-in-Time-Kompilierung zu reduzieren. Die gemessenen Ausführungszeiten werden in Millisekunden gemessen und am Ende jedes Backtestes angezeigt.

3.6 Abrufen der historischen Kursdaten

Die historischen Kursdaten werden über Binance bezogen. [10] Binance bietet eine API an, über die Finanzdaten für verschiedene Aktien oder Kryptowährungen in gewissen Zeiträumen abgerufen werden können. Die Daten werden aber nicht direkt weiterverwendet, sondern in einem Vorverarbeitungsschritt in das benötigte JSON Format umgewandelt. Dabei wird der durchschnittliche Kurswert pro Minutenintervall berechnet und zusammen mit dem Zeitstempel in die JSON Datei geschrieben. Mit der Normalisierung der Daten auf Minuten wird eine einheitliche Datenbasis für die Backtests geschaffen werden. Die Datei kann anschließend lokal gespeichert werden. Die verarbeiteten Datensets können in der Weboberfläche des Backtesters hochgeladen und verwendet werden.

3.7 Werkzeuge zur WASM Entwicklung

Für die Entwicklung von WebAssembly (WASM) können verschiedenste Werkzeuge eingesetzt werden. Ein Compiler ist notwendig, um den Quellcode in das WASM-Format zu übersetzen. Häufig verwendete Compiler sind dafür Emscripten für C/C++ [16, 58], TeaVM für Java [2, 55] oder rustc für Rust [17].

Des Weiteren können Tools wie das The WebAssembly Binary Toolkit (WABT) [59] zur Optimierung und Analyse von WASM-Modulen verwendet werden.

In dieser Arbeit wurde Emscripten als Compiler von C Code und WABT als Tool zur Auswertung von kompilierten WASM Dateien eingesetzt.

3.7.1 Emscripten

Emscripten ist ein Open Source Compiler, der C und C++ Code in WebAssembly (WASM) übersetzt.

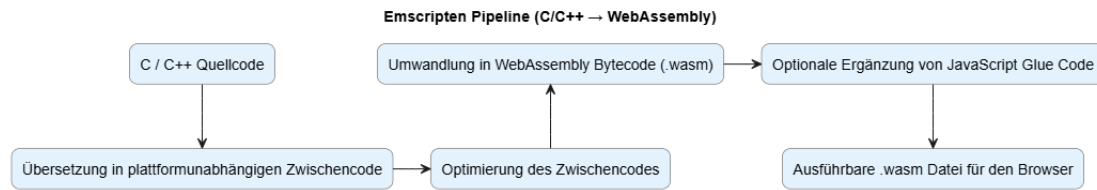


Abbildung 3.6: Emscripten Pipeline

In der Abbildung 3.6 ist der Ablauf eines Emscripten Kompiliervorgangs dargestellt. Zuerst wird der C/C++ Quellcode mit dem Clang Frontend in eine Zwischensprache übersetzt. Die Zwischensprache wird danach je nach Compileroption optimiert, um die Performance zu verbessern. Im nächsten Schritt entsteht aus der optimierte Zwischensprache WebAssembly Code. Abschließend kann optional JavaScript-Glue-Code generiert werden, der das Aufrufen des WebAssembly Moduls in JavaScript erleichtert. Dieser zusätzliche Code kann jedoch die Ausführungszeit des Moduls beeinträchtigen. [15]

Um die bestmögliche Performance zu erzielen, wird in dieser Arbeit auf die Generierung von JavaScript-Glue-Code verzichtet und mit der höchsten Optimierungsstufe O3 kompiliert. Somit wird gewährleistet, dass der erzeugte WebAssembly Code möglichst effizient ausgeführt werden kann.

Alle Strategien sind wie folgt kompiliert:

```
emcc strategie.c -O3 -nostdlib -Wl,-no-entry -Wl,-allow-undefined -o strategie.wasm
```

- **emcc**: Befehl zum Aufrufen von Emscripten.
- **strategie.c**: Die C-Quelldatei der jeweiligen Strategie.
- **-O3**: Die Optimierungsstufe.
- **-nostdlib**: Verhindert das Einbinden von Standardbibliotheken.
- **-Wl,-no-entry**: Gibt an, dass das Modul keinen Einstiegspunkt hat. In diesem Fall ist dies die fehlende main Funktion, da es sich um ein Modul handelt.
- **-Wl,-allow-undefined**: Erlaubt undefinierte Symbole im Modul.
- **-o strategie.wasm**: Legt den Namen der Ausgabedatei fest.

3.7.2 WABT

Das WebAssembly Binary Toolkit (WABT) bietet eine Zusammenstellung von Werkzeugen zur Untersuchung und nachträglichen Änderung von WebAssembly (WASM) Modulen. WASM Dateien können damit beispielsweise in ein menschenlesbares Textformat umgewandelt, dekompiert oder nachträglich verändert und validiert werden. Im lesbaren Textformat ist der Bytecode nicht mehr binär kodiert, sondern durch Assembler Code dargestellt, der leichter zu interpretieren ist. WABT bietet des weiteren Werkzeuge zur Darstellung in JSON oder einem Object Dump Format an. Ebenfalls ist es möglich Metainformationen über das Modul zu extrahieren, wie beispielsweise die Anzahl der Funktionen, den Speicherverbrauch oder die importierten und exportierten Symbole. [59] Der in der Abbildung gezeigte Bytecode wurde mit WABT decodiert. Eine Darstellung des Bytecodes ist in der Abbildung 2.4 zu sehen.

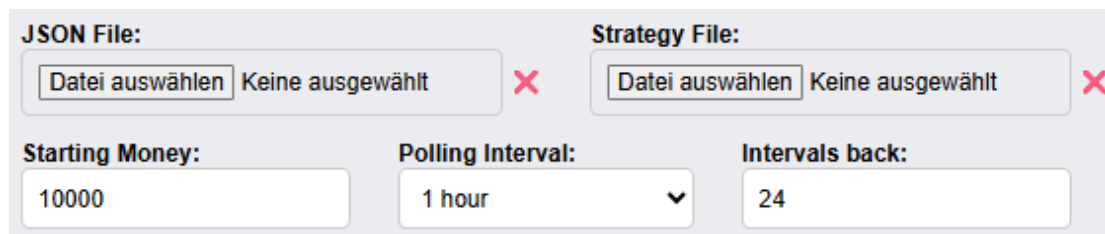
Kapitel 4

Implementierung des Backtesters und der Strategien

In diesem Kapitel wird beschrieben wie die einzelnen Komponenten des Backtester und die getesteten Strategien implementiert sind.

4.1 Die Benutzerschnittstellen des Backtesters

Die Benutzerschnittstelle des Backtesters ist in HTML und JavaScript implementiert. Der Backtester stellt insgesamt fünf Eingabekomponenten zur Parametrisierung des Backtests bereit.



The image shows a user interface for configuring backtest parameters. It consists of two rows of input fields. The first row has two sections: 'JSON File:' and 'Strategy File:'. Each section contains a button labeled 'Datei auswählen' and a text field showing 'Keine ausgewählt', followed by a red 'X' icon. The second row has three input fields: 'Starting Money:' with a text field containing '10000', 'Polling Interval:' with a dropdown menu showing '1 hour', and 'Intervals back:' with a text field containing '24'.

Abbildung 4.1: Eingabefelder der Backtestparameter

In der Abbildung 4.1 sind die Eingabefelder der Backtestparameter zu sehen. Über ein Dateieingabefeld können JavaScript- und WebAssembly-Dateien hochgeladen werden, welche die Handelsstrategie implementieren. Ein weiteres Dateieingabefeld dient zum Import einer JSON-Datei, die historische Kursdaten enthält. Darüber hinaus ermöglichen zwei numerische Eingabefelder die Festlegung des Startkapitals sowie der Anzahl der rückblickend zu berücksichtigenden Zeitintervalle. Ergänzend dazu erlaubt ein Auswahlfeld die Auswahl des gewünschten Abfrageintervalls für die Kursdaten. Mögliche Optionen für das Abfrageintervall sind eine Stunde, 30 Minuten, 15 Minuten oder eine Minute.

Falls eine der Eingaben ungültig ist, kann der Backtest nicht gestartet werden. Des Weiteren werden die Eingaben nur zu Beginn des Backtests ausgelesen, sodass Änderungen während eines laufenden Backtests keine Auswirkungen haben.

4.2 Laden und Vorverarbeitung der Kursdaten

Die historischen Kursdaten werden über die API der Kryptowährungsbörse Binance abgerufen. Die erhaltenen Kursdaten umfassen Open-, High-, Low- und Close-Preise sowie das Handelsvolumen zu festgelegten Zeitintervallen. [10] Für die Erstellung der JSON-Datei werden nicht die einzelnen Handelssereignisse berücksichtigt, sondern minütlich gruppierte Durchschnittswerte verwendet. Dies reduziert die Datenmenge erheblich und ermöglicht eine effizientere Verarbeitung während des Backtests.

Ebenfalls liefert Binance bei großen Datenmengen nicht die gesamten Daten auf einmal, sondern limitiert die Anzahl der zurückgegebenen Datensätze pro Anfrage. Daher muss die Datenabfrage mehrfach durchgeführt werden, um den gesamten Zeitraum abzudecken.

Die API ist öffentlich zugänglich, aber es gelten bestimmte Ratenbegrenzungen. Um diese Ratenlimitierung nicht zu überschreiten, ist eine Verzögerung zwischen aufeinanderfolgenden Anfragen eingefügt.

Listing 4.1: Aufruf der Binance API Schnittstelle

```
1  while (start < end) {
2      const url = new URL("https://api.binance.com/api/v3/klines");
3      url.searchParams.set("symbol", symbol);
4      url.searchParams.set("interval", interval);
5      url.searchParams.set("startTime", start);
6      url.searchParams.set("endTime", end);
7      url.searchParams.set("limit", 1000);
8
9      const res = await fetch(url);
10     const data = await res.json();
11
12     // Fehlerbehandlung
13
14     allData.push(...data);
15
16     start = data[data.length - 1][6] + 1;
17
18     await new Promise(r => setTimeout(r, 200));
19 }
```

In dem Codeausschnitt 4.1 ist zu sehen, wie die API aufgerufen wird. Entsprechend den in den Eingabeparametern getroffenen Einstellungen werden das Handelssymbol sowie der Zeitraum in die URL eingebunden. Zusätzlich wird der Parameter limit auf den Wert 1000 gesetzt, um die maximale Anzahl an Datenpunkten pro Anfrage zu erhalten. Danach wird die API aufgerufen und die erhaltenen Daten werden gespeichert. Im Anschluss wird der Startzeitpunkt für die nächste Anfrage auf den Zeitpunkt des letzten erhaltenen Datenpunktes plus eine Millisekunde gesetzt. Die Millisekunde wird hinzugefügt, um Überschneidungen der Datenpunkte zu vermeiden. Eine kurze Verzögerung von 200 Millisekunden wird eingefügt, um die Ratenbegrenzung der API nicht zu überschreiten. Dieser Prozess wiederholt sich, bis der Endzeitpunkt erreicht ist.

4.3 Laden der Strategien

In diesem Unterkapitel wird beschrieben, wie extern implementierte Strategien aus hochgeladenen Dateien geladen und aufrufbar gemacht werden. In dem Backtester werden JavaScript und WebAssembly Strategien unterstützt, um eine flexible und erweiterbare Architektur zu ermöglichen. Abhängig vom Dateityp wird die hochgeladene Datei entweder als JavaScript-Funktion oder als WebAssembly-Modul interpretiert.

Listing 4.2: Laden der Strategien aus den hochgeladenen Dateien

```
1  // Bei JavaScript-Dateien
2  const reader = new FileReader();
3  reader.onload = function (e) {
4      const scriptContent = e.target.result;
5      strategyJS = new Function('return ' + scriptContent)();
6
7      if (typeof strategyJS === 'function') {
8          uploadedFunctionAvailable = "js";
9      } else {
10         strategyJS = null;
11         uploadedFunctionAvailable = "";
12     }
13 };
14
15 reader.readAsText(file);
16
17 // Bei WebAssembly-Dateien
18 const wasmArrayBuffer = await file.arrayBuffer();
19 ({ strategy, memory } = await WebAssembly.instantiate(wasmArrayBuffer, {
20     env: {
21         abort: () => console.error('Abort called')
22     }
23 }).then(result => result.instance.exports));
24
25 if (strategy && memory) {
26     uploadedFunctionAvailable = "wasm";
27 } else {
28     uploadedFunctionAvailable = "";
29 }
```

Im Codeausschnitt 4.2 ist zu sehen, wie Strategie aus den unterschiedlichen Dateitypen instanziiert werden. Bei JavaScript-Dateien wird ein `FileReader` verwendet, der den Inhalt ausliest und in ein Funktionsobjekt umwandelt. Dazu wird der Inhalt der Datei mit einem `return` Statement kombiniert und in ein neues Funktionsobjekt zusammengefügt. Wenn das entstandene Objekt ohne Fehler in eine Funktion umgewandelt werden kann, wird dieses als Strategie gespeichert und es wird im System vermerkt, dass eine JavaScript-Strategie verfügbar ist.

Bei WebAssembly-Dateien wird der Inhalt der Datei in ein `ArrayBuffer` gegeben. In diesem `ArrayBuffer` ist der Code des WebAssembly-Moduls gespeichert. Anschließend wird das Modul mit der WebAssembly API instanziiert. Dabei werden der Buffer und ein Importobjekt übergeben, das eine `abort` Funktion bereitstellt, die von WebAssembly aufgerufen werden kann, um Fehler zu melden. Nach der Instanziierung werden alle exportierten Funktionen und Speicherbereiche des Moduls extrahiert und bereitgestellt. Wenn die erwarteten Exporte, Strategie Funktion und Speicherbereich, vorhanden sind,

werden diese im Backtester gespeichert und es wird vermerkt, dass eine WebAssembly-Strategie verfügbar ist.

4.4 Implementierung der Backtestlogik

Die Backtestlogik ist vollständig in JavaScript implementiert, während WebAssembly ausschließlich zur Ausführung der Handelsstrategien eingesetzt werden kann. Der Backtestingprozess iteriert über die historischen Kursdaten und ruft bei jedem Iterationsschritt die Handelsstrategie auf, um Kauf- oder Verkaufsentscheidungen zu treffen. Bei der Verarbeitung des Rückgabewerts der Strategie wird nicht berücksichtigt, in welcher Programmiersprache die Strategie implementiert ist. Unterschiede bestehen nur darin, wie die Kursdaten an die Strategie übergeben werden und ob vor einem Aufruf Speicher für WebAssembly reserviert werden muss.

Listing 4.3: Backtestlogik

```

1   if (uploadedFunctionAvailable === "wasm") {
2       array = new Float64Array(memory.buffer, 0, intervalsBack)
3   }
4
5   for (let i = 0; i < jsonData.length; i += interval) {
6       const startIdx = Math.max(0, i - intervalsBack * interval);
7       const sliced = jsonData.slice(startIdx, i);
8
9       const prices = [];
10      for (let idx = 0; idx < sliced.length; idx += interval) {
11          prices.push(parseFloat(sliced[idx].avgPrice));
12      }
13
14      let resCode = 0;
15      if (uploadedFunctionAvailable === "js") {
16          resCode = strategyJS(prices, intervalsBack);
17      } else if (uploadedFunctionAvailable === "wasm" && array) {
18          array.set(prices);
19          if (strategy) {
20              resCode = strategy(array.byteOffset, prices.length, intervalsBack);
21          }
22      } else {
23          return;
24      }
25
26      // Fehlerbehandlung der Strategieergebnisse
27      // Umwandlung des Rückgabewerts in ein Aktionsobjekt
28
29      if (action) {
30          if (action.signal === 'buy' && money > 0) {
31              rowNumber = appendTradeRow(time, action, rowNumber, date, price,
tbody);
32
33              stock = money / price;
34              money = 0;
35              holding = true;
36          } else if (action.signal === 'sell' && stock > 0) {

```

```
37         rowNum = appendTradeRow(time, action, rowNum, date, price,
tbody);
38
39         money = stock * price;
40         stock = 0;
41         holding = false;
42     }
43 }
44
45     const value = holding ? stock * price : money;
46     const lastVal = portfolioValues[portfolioValues.length - 1]?.value;
47
48     if (lastVal !== value) {
49         portfolioValues.push({ time, value });
50     }
51 }
```

Im Codeausschnitt 4.3 ist die Hauptschleife der Backtestlogik zu sehen. Vor dem Beginn der Schleife werden verschiedenste Initialisierungen vorgenommen und Umgebungsvariablen gesetzt. Falls die hochgeladene Strategie in WebAssembly implementiert ist, wird ein Float64Array erstellt, damit die Kursdaten an die Strategie übergeben werden können. Die Schleife iteriert über die Kursdaten in den voreingestellten Intervallen. Zuerst wird der Bereich der Kursdaten bestimmt, der an die Strategie übergeben werden soll. Für dieses Zeitfenster werden die Kurswerte herausgenommen und gespeichert. Je nach hochgeladener Strategie wird danach die Strategiefunktion in JavaScript oder WebAssembly aufgerufen. Der JavaScript Variante werden die Kurswerte im Zeitfenster und die Anzahl der zu berücksichtigten Intervalle direkt übergeben. Bei der WebAssembly Version werden die Kurswerte zuvor in das reservierte Array kopiert und die Byte-Offset-Adresse des Arrays zusammen mit der Länge der Kurswerte und der Anzahl der zu berücksichtigten Intervalle übergeben. Falls ein Fehler bei der Ausführung der Strategie auftritt, wird der Backtest abgebrochen. Anschließend wird der Rückgabewert der Strategie in ein Aktionsobjekt umgewandelt. Wenn eine Kauf- oder Verkaufsaktion zurückgegeben wird, wird diese ausgeführt und in der Benutzeroberfläche protokolliert. Verkaufssignale werden nur ausgeführt, wenn Bestände vorhanden sind, während Kaufsignale nur ausgeführt werden, wenn genügend Kapital vorhanden ist. Danach wird der aktuelle Portfoliowert neu berechnet und gespeichert. Dieser Vorgang wird wiederholt, bis alle Kursdaten verarbeitet wurden.

4.5 Implementierungen der Strategien

Es werden drei verschiedene Handelsstrategien implementiert. Jede Strategie hat eine unterschiedliche Logik, um Kauf- und Verkaufsentscheidungen zu treffen. Diese Unterschiede in der Logik führen zu unterschiedlichen Laufzeitkomplexitäten. Die Ideen hinter den implementierten Strategien sind in Kapitel 2.4 beschrieben.

Jede der folgenden Untersektionen beschreibt die Implementierung einer der drei Strategien in JavaScript und C.

4.5.1 Gleitende Durchschnittsüberschneidungs Strategie

Die gleitende Durchschnittsüberschneidungs Strategie erzeugt Handelssignale dadurch, dass zwei unterschiedlich lange gleitende Durchschnitte der Kursdaten berechnet und miteinander verglichen werden. Die Richtung, von der sich der kurzfristige und der langfristige Durchschnitt schneiden bestimmt, ob ein Kauf- oder Verkaufssignal generiert wird. Das Konzept der Strategie ist in Kapitel 2.4.2 genauer beschrieben.

Listing 4.4: Durchschnittsüberschneidung in C

```

1  // Initialisierungen
2
3  for (int i = prices_length - 1 - shortPeriod; i < prices_length - 1; i++) {
4      shortPrev += prices[i];
5  }
6  shortPrev /= shortPeriod;
7
8  for (int i = prices_length - 1 - longPeriod; i < prices_length - 1; i++) {
9      longPrev += prices[i];
10 }
11 longPrev /= longPeriod;
12
13 double shortCurr = 0.0;
14 double longCurr = 0.0;
15
16 for (int i = prices_length - shortPeriod; i < prices_length; i++) {
17     shortCurr += prices[i];
18 }
19 shortCurr /= shortPeriod;
20
21 for (int i = prices_length - longPeriod; i < prices_length; i++) {
22     longCurr += prices[i];
23 }
24 longCurr /= longPeriod;
25
26 const double diffPrev = shortPrev - longPrev;
27 const double diffCurr = shortCurr - longCurr;
28
29 double absDiff = diffCurr < 0.0 ? -diffCurr : diffCurr;
30
31 const double currentPrice = prices[prices_length - 1];
32 const double threshold = currentPrice * 0.002;
33
34 if (diffPrev < 0.0 && diffCurr > 0.0 && absDiff >= threshold) {
35     return 1;
36 }
37 if (diffPrev > 0.0 && diffCurr < 0.0 && absDiff >= threshold) {
38     return -1;
39 }

```

Im Codeausschnitt 4.4 ist die Implementierung der gleitenden Durchschnittsüberschneidungs Strategie in C dargestellt. Nach den Initialisierungen wird der kurze Durchschnitt für den vorherigen Zeitraums berechnet. Dazu werden alle Kurswerte vor dem aktuellen Zeitpunkt bis zur Länge des kurzen Zeitraums aufsummiert und durch die Länge des kurzen Zeitraums geteilt. In dieser Implementierung werden für das kurze Zeitintervall acht vergangene Kurswerte berücksichtigt. Anschließend wird der lange Durchschnitt für

den vorherigen Zeitraum auf die gleiche Weise berechnet. Für den längeren Zeitraum werden 24 vergangene Kurswerte verwendet. Die Wahl der Zeiträume ist in Kapitel 3.2.3 erklärt.

Um bestimmen zu können, ob eine Überschneidung der Durchschnitte stattgefunden hat, werden die Durchschnittswerte für beide Zeiträume unter Einbeziehung des aktuellsten Kurswertes erneut berechnet. Durch den Vergleich der Differenzen der Durchschnittswerte vor und mit dem aktuellen Kurswert kann festgestellt werden, ob eine Überschneidung stattgefunden hat. Zusätzlich wird überprüft, ob die Differenz der Durchschnittswerte einen bestimmten Schwellenwert überschreitet, um Fehlsignale zu vermeiden. Der Schwellenwert wird als 0.2% des aktuellen Kurswertes definiert. Abschließend wird basierend auf dem Vorzeichen der Differenzen entschieden, ob ein Kauf- oder Verkaufssignal generiert wird.

Listing 4.5: Durchschnittsüberschneidung in JavaScript

```
1  function movingAverageSlice(data, start, end) {
2      if (end <= start) {
3          return 0;
4      }
5      let sum = 0;
6      for (let i = start; i < end; i++) {
7          sum += data[i];
8      }
9      return sum / (end - start);
10 }
11
12 const shortPrev = movingAverageSlice(prices, prevShortStart, prevEnd);
13 const longPrev = movingAverageSlice(prices, prevLongStart, prevEnd);
14
15 const currShortStart = length - shortPeriod;
16 const currLongStart = length - longPeriod;
17
18 const shortCurr = movingAverageSlice(prices, currShortStart, length);
19 const longCurr = movingAverageSlice(prices, currLongStart, length);
```

In der JavaScript Implementierung 4.5 werden die gleitenden Durchschnitte mit einer Hilfsfunktion berechnet. Die Intervalle sind für die Berechnung der kurzen und langen Durchschnitte gleich wie in der C Implementierung. Die Logik zur Erzeugung von Haltesignalen ist ebenfalls identisch zur C Version. Der Unterschied in den Implementierungen besteht darin, dass die Berechnung der Durchschnitte in JavaScript in eine Funktion ausgelagert ist. In C könnte diese Auslagerung zu einem Geschwindigkeitsverlust führen, da Funktionsaufrufe zusätzlichen Overhead verursachen. Compiler-Optimierungen wie Inlining würden in diesem Fall nicht zuverlässig an dieser Stelle eine Optimierung durchführen können. [42]

4.5.2 Mittelwertrückkehr Strategie

Bei der Mittelwertrückkehr Strategie werden Handelssignale bestimmt, indem der aktuelle Kurswert mit den Grenzen eines Bollinger Bandes verglichen wird. Das Bollinger Band wird durch den Mittelwert und die Standardabweichung der Kurswerte definiert. Das Konzept der Strategie ist in Kapitel 2.4.1 beschrieben.

Listing 4.6: Mittelwertrückkehr in C

```
1  double sum = 0;
2  for (int i = 0; i < length; i++) {
3      sum += prices[i];
4  }
5  double mean = sum / length;
6
7  double var_sum = 0;
8  for (int i = 0; i < length; i++) {
9      double diff = prices[i] - mean;
10     var_sum += diff * diff;
11 }
12 double variance = var_sum / length;
13
14 double stdDev = sqrt(variance);
15
16 double upperBand = mean + 3.25 * stdDev;
17 double lowerBand = mean - 3.25 * stdDev;
18
19 double lastPrice = prices[length - 1];
20
21 if (lastPrice < lowerBand) {
22     return 1;
23 }
24 if (lastPrice > upperBand) {
25     return -1;
26 }
```

Im Codeausschnitt 4.6 ist die Implementierung der Mittelwertrückkehr Strategie in C zu sehen. Zuerst wird der Mittelwert der übergebenen Kurswerte berechnet. Dies wird manuell mit einer Schleife durchgeführt. Danach wird die Varianz berechnet, indem die Differenz jedes Kurswertes zum Mittelwert quadriert und aufsummiert wird. Dazu wird auch keine Hilfsfunktion verwendet, sondern eine Schleife. Anschließend wird die Standardabweichung als Quadratwurzel der Varianz berechnet. Das Bollinger Band wird berechnet, indem der Mittelwert um das 3.25-fache der Standardabweichung nach oben und unten erweitert wird. Der Faktor 3.25 wurde gewählt, um eine breitere Bandbreite zu erhalten und somit weniger Fehlsignale, auch bei sehr volatilen Kursen zu generieren. Dies reduziert die Anzahl der generierten Handelssignale und führt zu stabileren Handelsergebnissen. Zum Schluss wird der letzte Kurswert mit den Grenzen des Bollinger Bandes verglichen, um Kauf- oder Verkaufsentscheidungen zu treffen.

Listing 4.7: Mittelwertrückkehr in JavaScript

```
1  const mean = prices.reduce((acc, val) => acc + val, 0) / prices.length;
2
3  const variance = prices.reduce((acc, val) => acc + Math.pow(val - mean, 2), 0) /
    prices.length;
4  const stdDev = Math.sqrt(variance);
```

In der JavaScript Implementierung 4.7 besteht der Hauptunterschied zur C Variante darin, dass die Berechnung des Mittelwerts und der Varianz mit der reduce Methode durchgeführt wird. Es müssen keine expliziten Schleifen verwendet werden, sondern nur Hilfsfunktionen aufgerufen. Dies macht den Code kompakter und leichter lesbar. Die restliche Logik zur Berechnung der Standardabweichung und der Bollinger Bänder ist

identisch zur C Implementierung.

4.5.3 Ausbruchsstrategie

Die Ausbruchsstrategie erzeugt Handelssignale, indem der aktuelle Preis mit den Extrema eines historischen Zeitfensters verglichen wird. Durch ein Epsilon wird eine kleine Toleranzzone um die Extrema definiert, um Fehlsignale zu reduzieren. Die Idee der Strategie ist in Kapitel 2.4.3 beschrieben.

Listing 4.8: Ausbruchsstrategie in C

```

1  int windowLength = length - 1;
2
3  // Berechnung des Maximums und Minimums innerhalb des Fensters
4
5  double lastPrice = prices[length - 1];
6
7  double epsilon = 0.01;
8  double longTrigger = max * (1.0 + epsilon);
9  double shortTrigger = min * (1.0 - epsilon);
10
11  if (lastPrice > longTrigger) {
12      return 1;
13  }
14  if (lastPrice < shortTrigger) {
15      return -1;
16  }
```

In der Implementierung in C 4.8 wird für die Berechnung der Extrema eine Schleife verwendet, die das Maximum und Minimum innerhalb des definierten Fensters bestimmt. Diese Werte werden mit einem Epsilon skaliert, um kleine Fehlsignale zu vermeiden. Zum Schluss wird der aktuelle Preis mit den berechneten Triggerwerten verglichen, um Kauf- oder Verkaufsentscheidungen zu treffen.

Listing 4.9: Ausbruchsstrategie in JavaScript

```

1  const window = prices.slice(0, prices.length - 1);
2  const { min, max } = extrema(window);
3  const lastPrice = prices[prices.length - 1];
4
5  const epsilon = 0.01;
6  const longTrigger = max * (1 + epsilon);
7  const shortTrigger = min * (1 - epsilon);
8
9  if (lastPrice > longTrigger) {
10     return 1;
11 }
12 if (lastPrice < shortTrigger) {
13     return -1;
14 }
```

In dem Codeausschnitt 4.9 wird das historische Fenster im Vergleich zur C Variante mit der slice Methode extrahiert. Ebenso werden die Extrema mit einer Hilfsfunktion extrema berechnet, die das Minimum und Maximum eines Arrays zurückgibt. Die restliche Logik zur Generierung der Handelssignale ist identisch zur C Implementierung.

Beide Implementierungen der Strategie haben die idente Logik. Die Unterschiede liegen in der Art und Weise, wie die Extrema berechnet werden. Die C Implementierung verwendet eine explizite Schleife, während die JavaScript Variante eine Hilfsfunktion nutzt.

4.6 Die Datenvisualisierungen

Zur Datenvisualisierung werden Kurs- und Portfoliowerte in einem HTML-Canvas dargestellt. Um eine visuelle überladene Darstellung und eine geringe Performanz zu vermeiden, wird pro Diagramm jeweils nur ein Teil der verfügbaren Daten visualisiert. Die Auswahl der dargestellten Datenpunkte erfolgt dabei so, dass die Aussagekraft der Diagramme erhalten bleibt, während die Darstellungsdichte reduziert wird.

4.6.1 Visualisierung des Kurswertes

Für die Darstellung der Kurswerte wird eine Stichprobe an historischen Kursdaten ausgewählt. Wie im Kapitel 3.4 beschrieben, wird jeder n-te Punkt der Daten auf der Grafik dargestellt.

Listing 4.10: Kurswertvisualisierung

```
1  const stepSize = Math.max(Math.round(data.length / 800), 1);
2  const sampled = data.filter((_, i) => i \% stepSize === 0);
3
4  sampled.forEach((d, index) => {
5      const time = new Date(d.time).getTime();
6      const price = parseFloat(d.avgPrice);
7      const x = padding + ((time - minTime) / (maxTime - minTime)) * plotWidth;
8      const y = padding + plotHeight - ((price / maxPrice) * plotHeight * 0.9);
9
10     points.push({ x, y, price, time: d.time });
11     if (index === 0) {
12         ctx.moveTo(x, y);
13     } else {
14         ctx.lineTo(x, y);
15     }
16 });
```

Im Codeausschnitt 4.10 ist zu sehen, wie die Kurswerte visualisiert werden. Zuerst wird die Schrittgröße berechnet, um die Anzahl der darzustellenden Punkte auf maximal 800 zu begrenzen. Anschließend wird eine Stichprobe der Kursdaten erstellt, indem jeder n-te Punkt ausgewählt wird. Danach werden die x- und y-Koordinaten für jeden ausgewählten Datenpunkt berechnet. Die x Position wird mit der Zeit des Datenpunkts relativ zur Gesamtzeitspanne errechnet. Die y-Koordinate wird basierend auf dem Kurswert relativ zum maximalen Kurswert skaliert. Schließlich werden die Punkte mit einer verbindenden Linie auf den Canvas gezeichnet.

4.6.2 Visualisierung des Portfoliowertes

Zur Darstellung des Portfoliowertes wird ebenfalls eine Stichprobe der berechneten Portfoliowerte verwendet. Für die Reduktion der Datenpunkte wird ebenfalls jeder nur jeder

n-te Punkt ausgewählt, um eine übersichtliche Darstellung zu gewährleisten.

Listing 4.11: Portfoliovisualisierung

```
1  const stepSize = Math.max(Math.round(portfolioValues.length / 800), 1);
2
3  for (let i = 0; i < portfolioValues.length; i += stepSize) {
4      const p = portfolioValues[i];
5      const x = padding + ((new Date(p.time).getTime() - minTime) / (maxTime -
minTime)) * plotWidth;
6      const y = padding + plotHeight - ((p.value / maxValue) * plotHeight);
7
8      if (i === 0) {
9          ctx.moveTo(x, y);
10     }
11     else {
12         ctx.lineTo(x, y);
13     }
14 }
```

Der Codeausschnitt 4.11 zeichnet die reduzierten Portfoliowerte auf einen Canvas. Ebenfalls wird in dieser Implementierung die Schrittgröße berechnet, um die Anzahl der darzustellenden Punkte auf maximal 800 zu begrenzen. Die Berechnung der Koordinaten der Datenpunkte erfolgt gleich wie zur Kurswertvisualisierung. Der Portfoliowert wird auch als Linie dargestellt, die die einzelnen Punkte miteinander verbindet.

Kapitel 5

Performanztests der unterschiedlichen Programmiersprachen

In diesem Kapitel werden die Rahmenbedingungen der Tests beschrieben, die durchgeführten Performanztests analysiert und die Ergebnisse interpretiert.

5.1 Die Testvoraussetzungen

Um die Ergebnisse der Performanztests vergleichen zu können, müssen die Tests unter den gleichen Voraussetzungen durchgeführt werden. Deswegen wird in diesem Abschnitt beschrieben, welche Voraussetzungen für die Tests geschaffen wurden.

5.1.1 Die Testumgebung

Alle Performanztests werden auf einem System mit der folgender Hardware und Software durchgeführt:

- CPU: Intel Core i7-11800H @2.30GHz
- RAM: 16 GB DDR4
- GPU: NVIDIA GeForce RTX 3070
- Betriebssystem: Windows 11 Pro 64-Bit
- Chrome Version: 143.0.7499.41
- Firefox Version: 145.0

Während der Durchführung der Tests werden keine weiteren Programme ausgeführt, um eine Verzerrungen der Ergebnisse zu vermeiden. Als Testumgebungen dienen die Browser Google Chrome und Mozilla Firefox, die jeweils die JavaScript Engines V8 (Chrome) [50] und SpiderMonkey (Firefox) [43] verwenden. Der Backtester wird in den beiden Browsern ausgeführt, um einen Vergleich der Performanz von WebAssembly (WASM) und JavaScript in der jeweiligen Engine zu ermöglichen.

5.1.2 Die Testdaten

Alle Strategien, die in den Performanztests getestet werden, verwenden die gleichen historischen Kursdatensätze. Für die Tests werden Kursdaten von zwei verschiedenen

Kryptowährungen verwendet:

- Bitcoin (BTC)
- Cardano (ADA)

Diese Kursdaten werden von der Kryptobörse Binance [10] bezogen und umfassen jeweils einen Zeitraum von einem und drei Jahren. Für die Testfälle wird Bitcoin im Zeitraum vom 01.07.2023 bis zum 30.06.2025 und Cardano im Zeitraum vom 01.07.2021 bis zum 30.06.2022. Die Kursdaten umfassen etwa 500.000 Datenpunkte pro Jahr und liegen in einem Minutenintervall vor. Somit stehen für Bitcoin rund 1,5 Millionen und für Cardano etwa 500.000 Kurswerte zur Verfügung. Durch die Kombination der Beobachtungszeiträume und Kryptowährungen wird ein breites Spektrum an unterschiedlichen Kursschwankungen abgebildet, sodass verschiedene Marktphasen und stark variierende Datenpunktvolumina in den Tests berücksichtigt werden.

```
[
  {
    "time": "yyyy-MM-ddThh:mm:ss.mssZ",
    "avgPrice": "xyz.xy"
  },
  {
    "time": "yyyy-MM-ddThh:mm:ss.mssZ",
    "avgPrice": "xyz.xy"
  }
]
```

Abbildung 5.1: Beispiel eines Kursdatensatzes im JSON-Format

Die Kursdaten werden im JSON-Format gespeichert, wie in Abbildung 5.1 dargestellt. Die Datenpunkte enthalten einen Zeitstempel und einen Kurswert. Die Daten sind im Minutentakt erfasst, was eine detaillierte Analyse der Kursbewegungen ermöglicht.

5.2 Die Testausführung

Damit die Performanz der beiden Programmiersprachen objektiv bewertet werden kann, werden die Ausführungszeiten der einzelnen Strategien systematisch gemessen. Es wird jede der drei Handelsstrategien sowohl in WebAssembly als auch in JavaScript implementiert und anschließend innerhalb des entwickelten Backtesters ausgeführt. Dafür wird das System jeweils einmal in Chrome und einmal in Firefox gestartet. Der Backtester garantiert dabei, dass alle Strategien unter gleichen Bedingungen getestet werden können.

Bei jedem Durchlauf erhält die Strategie den aktuellen Datenpunkt sowie die letzten 72 historischen Datenpunkte. Auf dieser Basis wird die Strategie im Minutenintervall abgefragt, um ein Handelssignal zu generieren. Dieses Signal wird anschließend ausgeführt, sofern in der Simulation ausreichend Geld oder Wertpapiere zur Verfügung stehen.

Um die Aussagekraft der Ergebnisse weiter zu erhöhen, werden alle Strategien auf zwei unterschiedliche Datensets angewandt. Durch die Kombination aus zwei Browsern, drei Strategien, zwei Implementierungssprachen und zwei Datensets ergeben sich insgesamt 24 Testfälle.

Für statistisch belastbare und reproduzierbare Ergebnisse wird jede Kombination aus Browser, Strategie, Implementierungssprache und Datenset 25-mal ausgeführt. Bei jedem Durchlauf wird die benötigte Zeit erfasst. Aus den Messdaten wird der Durchschnitt, der Median und die Standardabweichung pro Testfall berechnet. Diese Kennzahlen ermöglichen es, sowohl die Performanz als auch die Stabilität und Konsistenz der jeweiligen Implementierung zu bewerten.

Untersuchungsmerkmal	Ausprägung
Browser	Chrome, Firefox
Handelsstrategien	Mittelwertrückkehr, Ausbruch, Durchschnittsüberschneidung
Implementierungssprachen	JavaScript, WebAssembly
Datensätze	0,5 Millionen Datenpunkte, 1,5 Millionen Datenpunkte
Wiederholungen pro Testfall	25
Zeitintervall	1 Minute
Historische Datenpunkte	72
Gemessene Kennzahlen	Mittelwert, Median, Standardabweichung
Anzahl Testfälle gesamt	$2 \times 3 \times 2 \times 2 = 24$

Tabelle 5.1: Übersicht der Rahmenbedingungen der Performanzmessung

In der Tabelle 5.1 sind die Rahmenbedingungen der Performanzmessung zusammengefasst.

5.3 Analyse der Testergebnisse

Ziel der Analyse ist es, die Laufzeitunterschiede zwischen der Ausführung des Backtesters mit reinem JavaScript und der Ausführung unter Verwendung von WebAssembly darzustellen und zu analysieren. Die Analyse konzentriert sich dabei auf die Interpretation der gemessenen Laufzeiten und deren erkennbaren Unterschiede. Speicherverbrauch und Größe der einzelnen Strategien werden in diesem Kontext nicht näher betrachtet.

5.3.1 Ergebnisse in Firefox

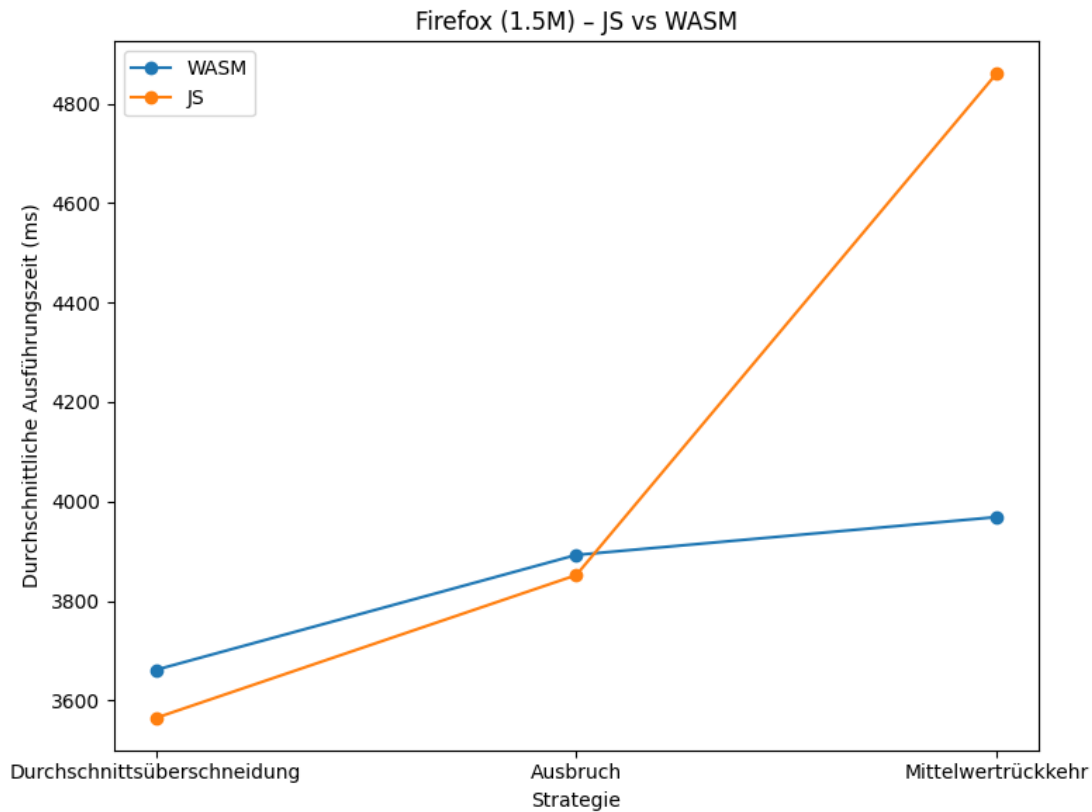


Abbildung 5.2: Durchschnittliche Ausführungszeiten der Strategien in Firefox mit 1,5 Millionen Datenpunkten

In Abbildung 5.2 ist zu sehen wie sich die Ausführungsdauer der drei Strategien bei 1,5 Millionen Datenpunkten in Firefox verhält. Dabei ist erkennbar, dass im Firefox Browser der Performanzvorteil von WebAssembly gegenüber JavaScript mit steigender Rechenintensität der Strategien zunimmt. Der durch WebAssembly erzielte Performanzvorteil kompensiert den zusätzlichen Overhead aber erst bei der rechenintensiven Strategien. Die Durchschnittsüberschneidungsstrategie weist den geringsten Rechenaufwand auf und benötigt entsprechend die kürzeste Berechnungszeit. In diesem Fall verringert der durch die Nutzung von WebAssembly entstehende Overhead die Performanz so sehr, dass die in JavaScript implementierte Strategie schneller ist.

Bei der Ausbruchsstrategie ist der Berechnungsaufwand höher, was zu einer Annäherung der Laufzeiten beider Implementierungen führt. WebAssembly erreicht hier nahezu vergleichbare Ausführungszeiten wie JavaScript.

Die Strategie der Mittelwertrückkehr unter Verwendung von Bollinger-Bändern erfordert den höchsten Rechenaufwand. In diesem Szenario zeigt sich ein deutlicher Performanzvorteil von WebAssembly. Die JavaScript Implementierung ist im Durchschnitt etwa 20% langsamer als die WebAssembly Variante.

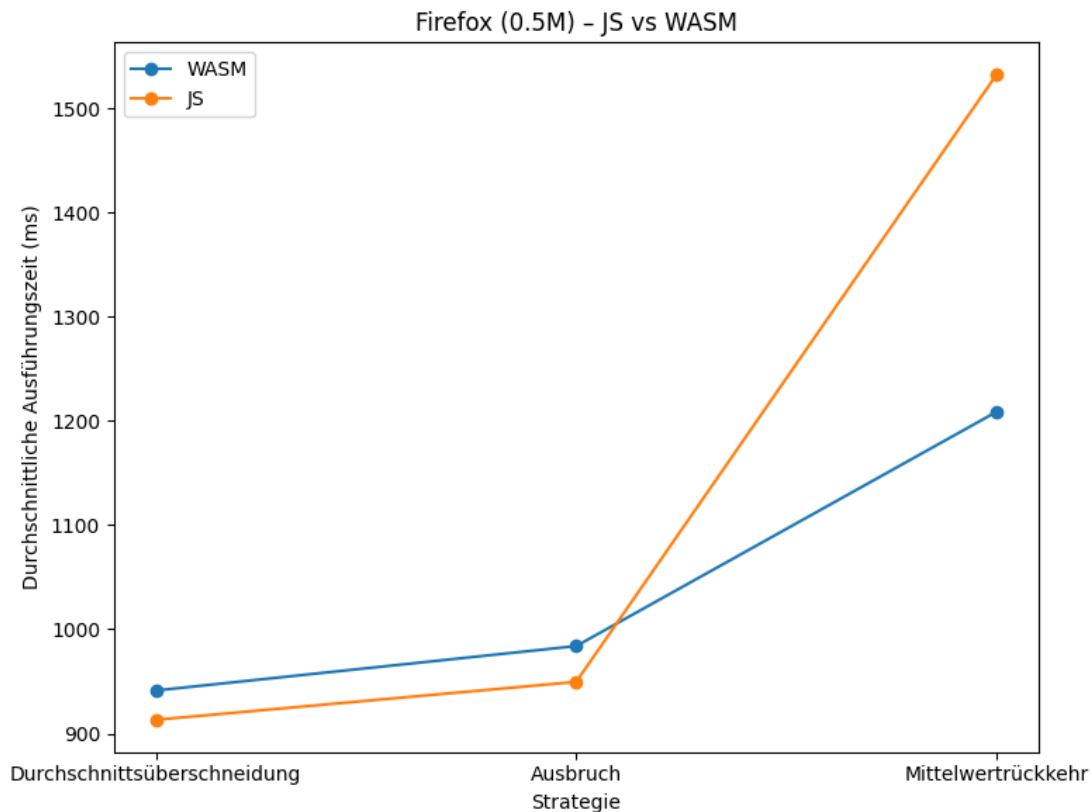


Abbildung 5.3: Durchschnittliche Ausführungszeiten der Strategien in Firefox mit einer halben Million Datenpunkten

In Abbildung 5.3 zeigt sich, ähnlich wie in Abbildung 5.2, dass der WebAssembly Overhead auch bei geringerer Datenpunktzahl erst bei rechenintensiven Strategien durch die gesteigerte Performanz kompensiert wird. Auffällig ist jedoch, dass in diesem Testfall eine Verdreifachung der Datenpunkte im Vergleich zum Szenario mit weniger Datenpunkten zu einer überproportionalen Erhöhung der Ausführungszeit führt. Bei 1,5 Million Datenpunkten dauert die durchschnittliche Ausführung der Strategien etwa 3,5x so lange wie bei einer halben Million Datenpunkten. Ein möglicher Grund für die Verringerung der Performanz bei vielen Datenpunkten kann darin liegen, dass mit steigender Anzahl an Datenpunkten auch der benötigte Laufzeitspeicher zunimmt. Der erhöhte Speicherbedarf führt zu einem entsprechend höheren Aufwand für die Speicherverwaltung im System. Insbesondere bei speicherintensiven Operationen kann dieser Effekt die Effizienz der Strategie deutlich verringern.

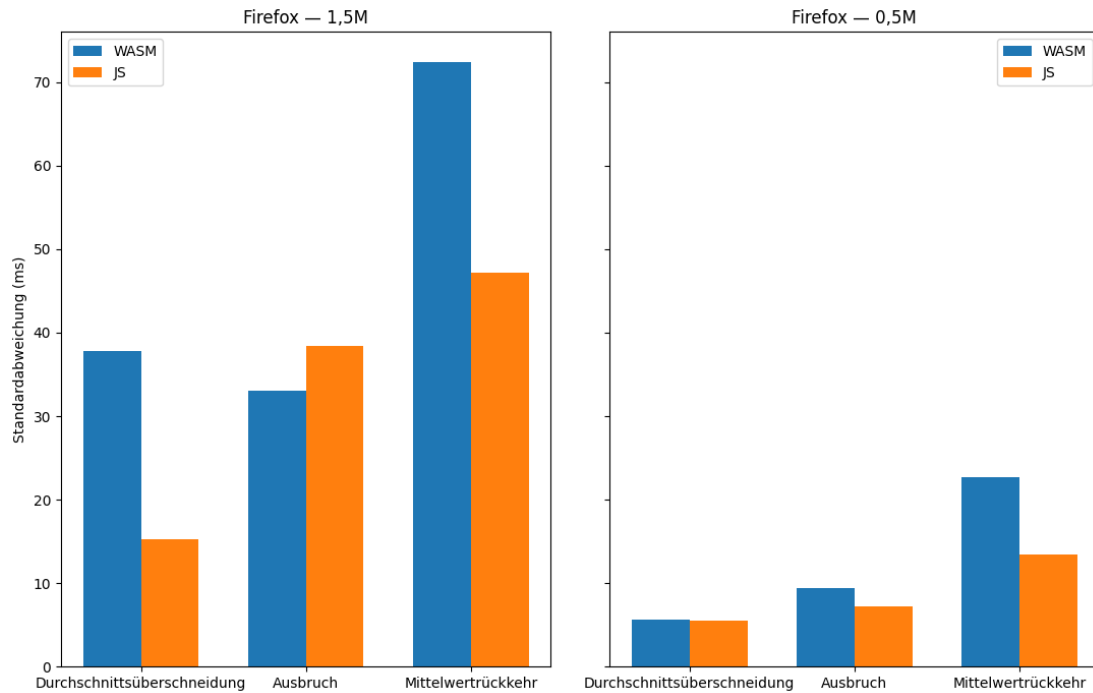


Abbildung 5.4: Standardabweichung der Strategien in Firefox

In Abbildung 5.4 ist die Standardabweichung der Ausführungszeiten der einzelnen Strategien in Firefox dargestellt. In beiden betrachteten Testfällen ist ein Anstieg der Standardabweichung mit zunehmendem Rechenaufwand zu erkennen. Dies bedeutet, dass rechenintensivere Strategien eine weniger vorhersehbare Ausführungszeit aufweisen. Besonders deutlich zeigt sich dieser Effekt bei der Mittelwertrückkehrstrategie mit 1,5 Millionen Datenpunkten in WebAssembly, für die die höchste Standardabweichung der Ausführungszeiten gemessen wurde.

Des Weiteren ist zu erkennen, dass die WebAssembly Implementierungen durchschnittlich eine höhere Standardabweichung aufweisen als die JavaScript Varianten. Der größte Unterschied ist bei der Durchschnittsüberschneidungsstrategie mit 1,5 Millionen Datenpunkten zu erkennen, bei dieser weist die WebAssembly Implementierung eine 2,5 Mal höhere Varianz in den Ausführungszeiten auf. Nur bei der Ausbruchsstrategie mit 1,5 Millionen Datenpunkten ist die Standardabweichung der WebAssembly Implementierung geringer als die der JavaScript Variante.

Für die beiden Strategien mit dem geringsten Rechenaufwand mit 500.000 Datenpunkten sind hingegen keine großen Unterschiede zwischen den Implementierungen erkennbar. Die Standardabweichungen sind in beiden Fällen miteinander vergleichbar.

5.3.2 Ergebnisse in Chrome

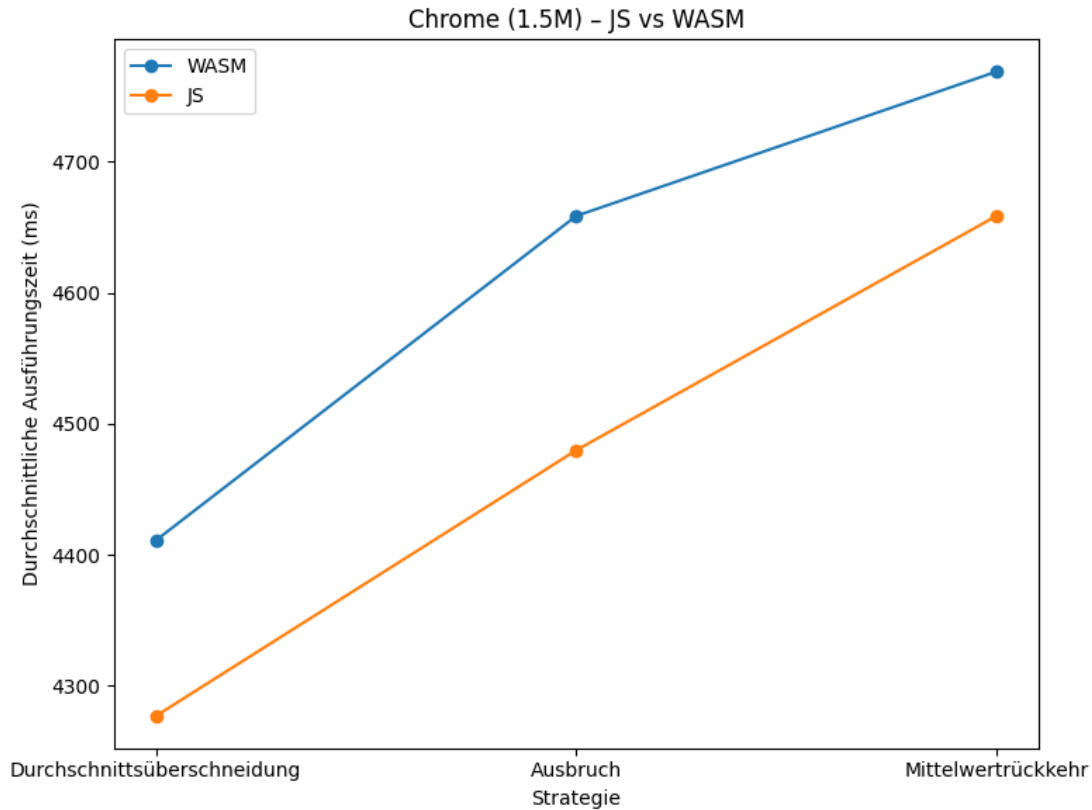


Abbildung 5.5: Durchschnittliche Ausführungszeiten der Strategien in Chrome mit 1,5 Millionen Datenpunkten

In Abbildung 5.5 sind die Ausführungszeiten der drei Strategien in Chrome bei 1,5 Millionen Datenpunkten zu sehen. Diese Testergebnisse zeigen das WebAssembly nicht so effizient wie JavaScript in Chrome ausgeführt wird. In Chrome ist WebAssembly bei allen drei Strategien langsamer als die JavaScript Implementierung, egal bei welchem Rechenaufwand der Strategie. In diesem Fall ist der Unterschied zwischen den beiden Implementierungssprachen über alle Strategien hinweg vergleichbar und beträgt etwa 3% bis 5% der Ausführungszeit. Dies deutet darauf hin, dass der Overhead von WebAssembly in der V8-Engine höher ist. WebAssembly profitiert in Chrome nicht von der gesteigerten Performanz bei rechenintensiven Aufgaben, wie dies in Firefox der Fall ist. Die Tests von [52] kommen ebenfalls bei Desktop-Anwendungen zu dem Ergebnis, dass die WebAssembly Performanz in Chrome schlechter ist als in Firefox.

Des Weiteren zeigt sich, dass die JavaScript Umsetzung der rechenintensiven Mittelwertrückkehrstrategie in Chrome schneller ausgeführt wird als in Firefox. Bei den beiden weniger rechenintensiven Strategien hingegen weist Chrome sowohl in WebAssembly als auch in JavaScript eine höhere Ausführungszeit auf. Folglich hat Chrome bei rechenintensiven JavaScript Anwendungen eine bessere Performanz als Firefox.

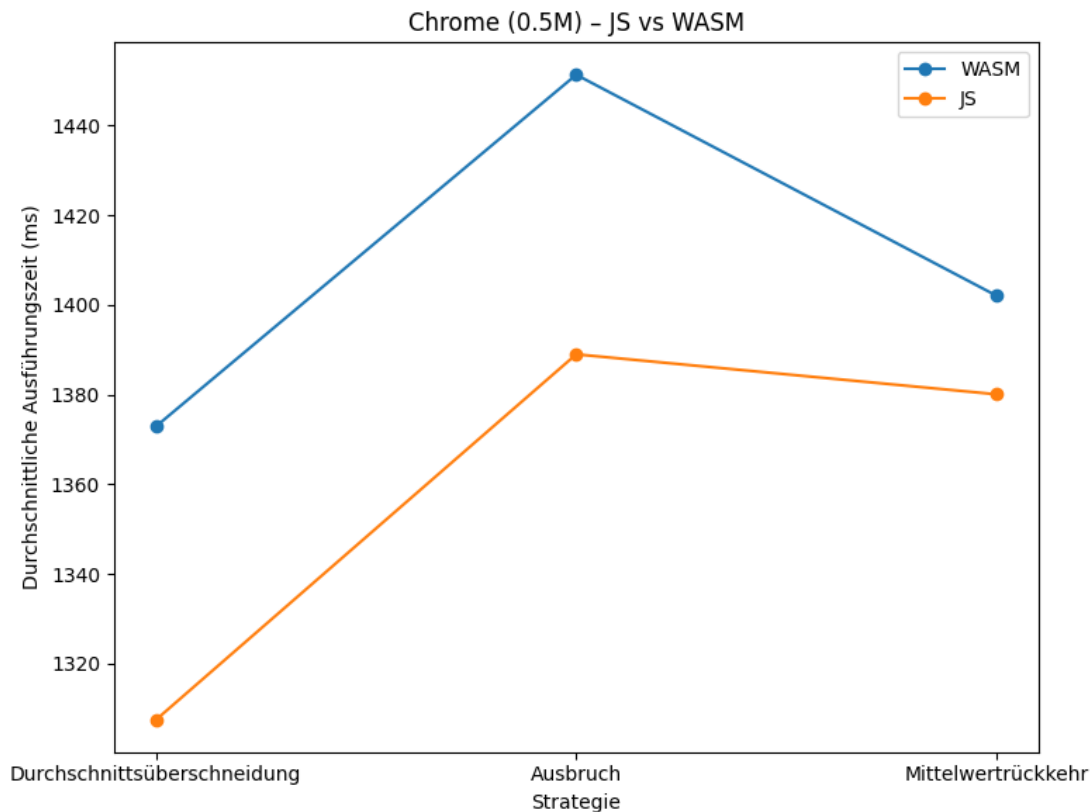


Abbildung 5.6: Durchschnittliche Ausführungszeiten der Strategien in Chrome mit einer halben Million Datenpunkten

In Abbildung 5.6 ist sichtbar, dass die Performanzunterschiede zwischen WebAssembly und JavaScript in Chrome auch bei geringerer Datenpunktzahl bestehen bleiben. Die WebAssembly Implementierungen sind weiterhin langsamer als die JavaScript Varianten, unabhängig vom Rechenaufwand der Strategie. Interessanterweise benötigt die Ausbruchsstrategie bei geringerer Datenpunktzahl am längsten zur Ausführung. Bei allen anderen Testfällen war die Mittelwertrückkehrstrategie die Zeitaufwendigste. Ein möglicher Grund hierfür könnte sein, dass bei dieser Funktion bei dieser Datenmenge noch keine Just-In-Time-Optimierungen durchgeführt wurden, die bei größeren Datenmengen zu einer schnelleren Ausführung führen könnten.

Der Effekt der überproportionalen Steigerung der Ausführungszeit bei erhöhter Datenpunktzahl, der in Firefox beobachtet wurde, ist in Chrome auch in diesem Testfall erneut erkennbar. Bei einer Verdreifachung der Datenpunkte von 500.000 auf 1,5 Millionen steigt die durchschnittliche Ausführungszeit der Strategien um etwa das Dreifache an.

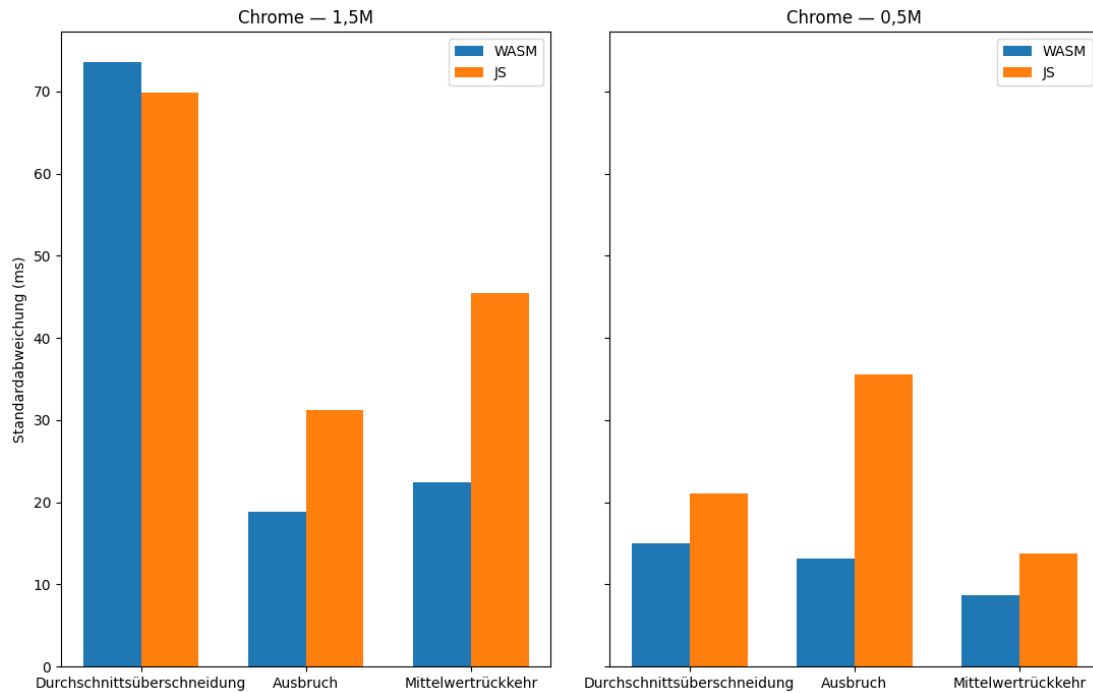


Abbildung 5.7: Standardabweichung der Strategien in Chrome

Abbildung 5.7 zeigt die Standardabweichung der Ausführungszeiten der einzelnen Strategien in Chrome. Im Gegensatz zu Firefox ist in Chrome kein klarer Trend erkennbar, dass die Standardabweichung mit steigendem Rechenaufwand zunimmt. Bei der Ausbruchsstrategie ist die Standardabweichung bei einer geringeren Anzahl von Datenpunkten sogar höher als bei einer größeren Datenmenge. Dies könnte darauf hindeuten, dass in Chrome externe Einflüsse wie Laufzeitoptimierungen in der JavaScript Engine einen stärkeren Einfluss auf die Messungen haben.

Des Weiteren im Gegensatz zu Firefox sind die Standardabweichungen der WebAssembly Implementierungen in Chrome überwiegend geringer als die der JavaScript Varianten. Nur bei der Durchschnittsüberschneidungsstrategie mit 1,5 Millionen Datenpunkten weist die WebAssembly Implementierung eine höhere Varianz auf als die JavaScript Version. Eine mögliche Erklärung hierfür ist eine stabilere Integration von WebAssembly in die Chrome Laufzeitumgebung, die zu geringeren Schwankungen der Ausführungszeiten führt.

Ebenfalls lässt sich in Chrome kein deutlicher Unterschied in der Standardabweichung der WebAssembly Implementierungen zwischen den beiden Datenmengen erkennen. Alle in WebAssembly implementierten Strategien außer der Durchschnittsüberschneidungsstrategie mit 1,5 Millionen Datenpunkten weisen eine ähnliche Standardabweichung auf. Dies deutet darauf hin, dass die zeitliche Stabilität primär durch die Kombination an Strategie und Datenmenge bestimmt wird.

5.3.3 Browserabhängige Performanz von WebAssembly im Vergleich

Browser	Durchschnittsüberschneidung	Ausbruch	Mittelwertrückkehr
Firefox	3661,52	3892,48	3968,64
Chrome	4411,00	4658,52	4768,64

Tabelle 5.2: Vergleich der Ausführungszeiten von WebAssembly in den beiden Browsern

In der Tabelle 5.2 sind die durchschnittlichen Ausführungszeiten der WebAssembly Implementierungen in den beiden Browsern zusammengefasst. Es ist erkennbar, dass Firefox in allen drei Strategien eine bessere Performanz aufweist als Chrome. Der Unterschied in den Ausführungszeiten zwischen den beiden Browsern ist bei allen Strategien vergleichbar und liegt bei circa 800 Millisekunden. Dies deutet darauf hin, dass die WebAssembly Integration in Firefox mit SpiderMonkey effizienter ist als in Chrome mit V8.

5.3.4 Analyse der WebAssembly Performanz ohne Berücksichtigung der Speicherverwaltung

Ein Teil des Overheads von WebAssembly resultiert aus der Speicherverwaltung, die sowohl bei der Initialisierung als auch während der Ausführung der Strategien anfällt. Um die Performanz von WebAssembly ohne den Einfluss der Speicherverwaltung zu analysieren, wird erneut die Ausführungszeit der Strategien gemessen. Dabei wird aber die Speicherverwaltung nicht mitberücksichtigt.

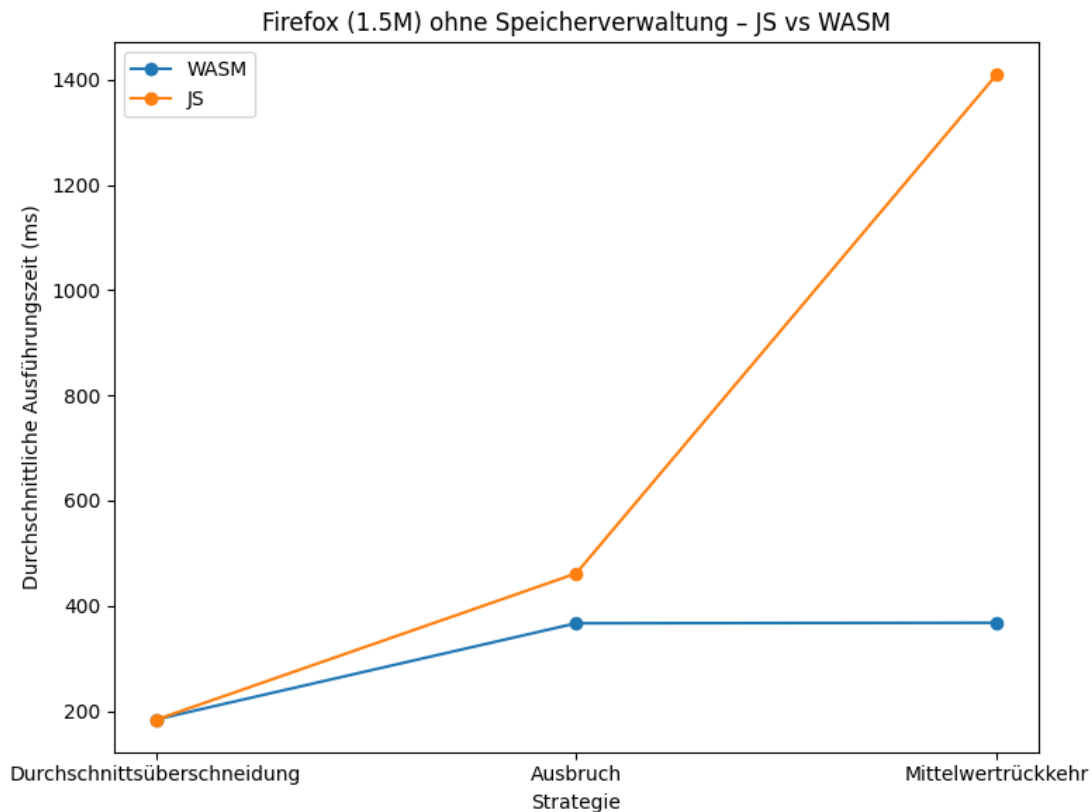


Abbildung 5.8: Durchschnittliche Ausführungszeiten der Strategien ohne Speicherverwaltung in Firefox mit 1,5 Millionen Datenpunkten

In Abbildung 5.8 ist zu sehen, wie sich die Ausführungszeiten der Strategien in Firefox verhalten, wenn die Speicherverwaltung nicht berücksichtigt wird. Im Vergleich zu den Ergebnissen in Abbildung 5.2 sind die WebAssembly Implementierungen der drei Strategien alle schneller als die JavaScript Varianten. Dies zeigt, dass die Speicherverwaltung einen Einfluss auf die Performanz von WebAssembly in Firefox hat. Dieser Einfluss ist groß genug, um alle WebAssembly Strategien performanter als die JavaScript Implementierungen zu machen.

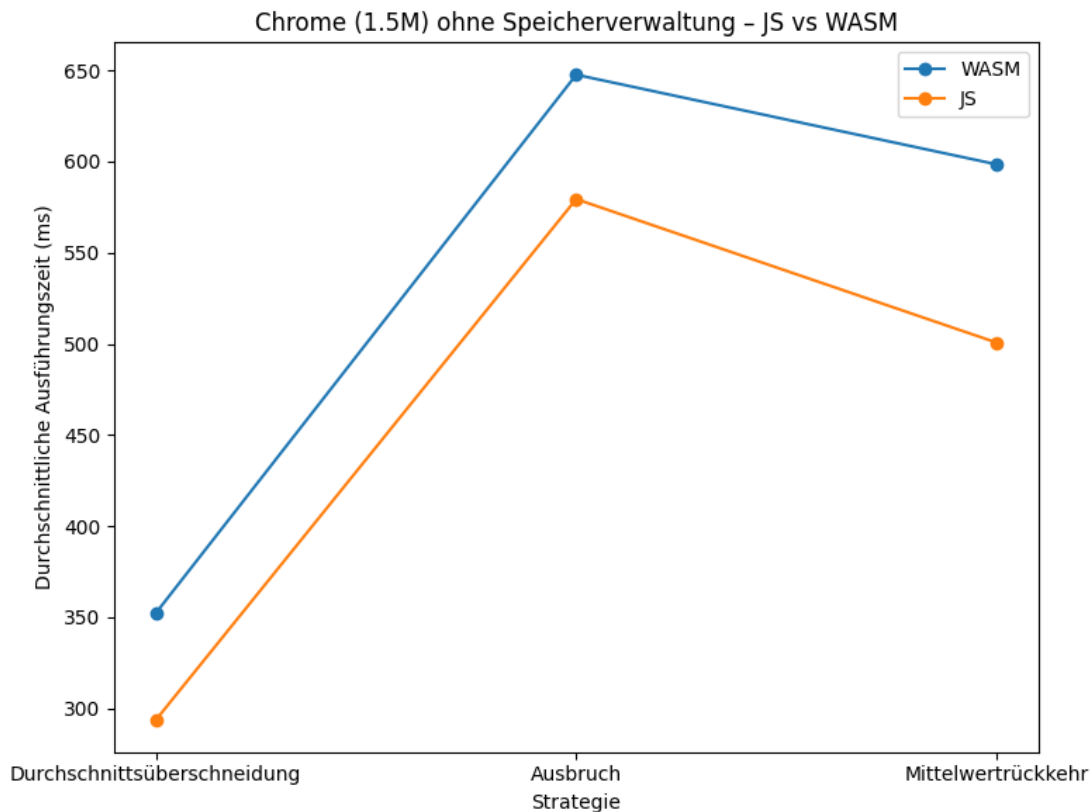


Abbildung 5.9: Durchschnittliche Ausführungszeiten der Strategien ohne Speicherverwaltung in Chrome mit 1,5 Millionen Datenpunkten

Die Abbildung 5.9 zeigt die Ausführungszeiten der Strategien in Chrome ohne Berücksichtigung der Speicherverwaltung. Auch in diesen Testfällen sind die JavaScript Implementierungen aller drei Strategien schneller als die WebAssembly Varianten. Dies war auch in den ursprünglichen Testergebnissen in Abbildung 5.5 der Fall. Der Unterschied besteht aber darin, dass die Performanzunterschiede zwischen den beiden Implementierungssprachen geringer sind. Mit Berücksichtigung der Speicherverwaltung war WebAssembly um circa 150 Millisekunden langsamer als JavaScript. Ohne der Speicherverwaltung beträgt der Unterschied durchschnittlich nur noch etwa 80 Millisekunden. Diese Testergebnisse verdeutlichen, dass die Speicherverwaltung auch in Chrome einen Einfluss auf die Performanz von WebAssembly hat, aber nicht genug, um den Overhead Nachteil gegenüber JavaScript zu kompensieren.

5.4 Zusammenfassung der Testergebnisse

In diesem Kapitel wurden Performanztests durchgeführt, um das Laufzeitverhalten von JavaScript- und WebAssembly Implementierungen innerhalb eines webbasierten Backtesters zu vergleichen. Die Tests erfolgten unter kontrollierten und reproduzierbaren Rahmenbedingungen, wobei unterschiedliche Browser, JavaScript und WebAssembly Engines, Handelsstrategien sowie Datensatzgrößen berücksichtigt wurden.

Die Ergebnisse zeigen, dass die Performanz von WebAssembly im Vergleich zu JavaScript stark von der verwendeten Laufzeitumgebung und der Rechenintensität der jeweiligen Strategie abhängt. In Firefox konnte WebAssembly bei der rechenintensiven Strategie einen Performanzvorteil gegenüber JavaScript erzielen. Bei den einfacheren Strategien erwies sich jedoch JavaScript aufgrund geringerem Initialisierungs- und Integrationsaufwand als effizienter.

In Chrome zeigte sich bei allen getesteten Strategien ein konsistenter Performanznachteil von WebAssembly gegenüber JavaScript. Dieser Nachteil blieb unabhängig von der Komplexität der Strategie und der Datenmenge bestehen. Dadurch lässt sich vermuten, dass bei diesem Anwendungsfall in Chrome WebAssembly weniger optimal in die V8-Engine integriert, ist als reines JavaScript.

Darüber hinaus wurde festgestellt, dass eine Erhöhung der Datenpunktzahl nicht linear zu einer Verlängerung der Ausführungszeit führt. Vielmehr zeigte sich in beiden Browsern, dass größere Datensätze zu einer überproportionalen Steigerung der Ausführungsdauer führen können.

Die Analyse der Standardabweichungen zeigte zudem, dass beide Browser unterschiedliche Stabilitätsprofile aufweisen. In Firefox neigen WebAssembly Implementierungen zu höheren Schwankungen in den Ausführungszeiten, während in Chrome JavaScript eine größere Varianz aufweist. Insgesamt zeigt sich, dass WebAssembly in Chrome browserübergreifend eine stabilere Ausführungszeit aufweist, unabhängig von der gewählten Strategie oder der Datenmenge.

Die reinen Ausführungszeiten ohne Berücksichtigung der Speicherverwaltung verdeutlichen, dass die Speicherverwaltung einen signifikanten Einfluss auf die Performanz von WebAssembly hat. In Firefox konnte WebAssembly ohne diesen Overhead alle JavaScript Implementierungen übertreffen. In Chrome blieb JavaScript jedoch auch ohne diese Berücksichtigung schneller als WebAssembly, jedoch verringerte sich der Performanzunterschied.

Zusammenfassend lässt sich feststellen, dass WebAssembly in Firefox und bei komplexen, rechenintensiven Strategien einen messbaren Performanzvorteil bieten kann. Während JavaScript in Chrome bei aufwendigen Aufgaben die bessere Laufzeitperformanz aufweist.

Kapitel 6

Zusammenfassung und Schlussbemerkungen

In diesem Kapitel wird die Arbeit zusammengefasst, ein Ausblick auf mögliche zukünftige Erweiterungen und Verbesserungen des Backtesters gegeben.

6.1 Zusammenfassung

In dieser Arbeit wurde ein clientseitiger Backtester mit Weboberfläche entwickelt, der den Vergleich von JavaScript und WebAssembly ermöglicht. Dieser wurde entwickelt, um die Performanzeunterschiede zwischen den beiden Programmiersprachen bei einer realistischen Anwendung zu untersuchen. Es wurden unterschiedlich rechenintensive Handelsstrategien implementiert und in den Backtester integriert. Dadurch kann die Ausführungsdauer auch bei steigender Komplexität der Strategien verglichen werden. Es gibt bereits clientseitige Backtester, die selbst implementierte Strategien unterstützen. Diese sind jedoch für den Vergleich der beiden Sprachen ungeeignet, da JavaScript und WebAssembly-Strategien nicht gleichzeitig unterstützt werden und somit kein direkter Vergleich möglich ist.

Der Backtester wurde in JavaScript implementiert. Die historischen Kursdaten können entweder als JSON-Datei hochgeladen oder mit der Binance API abgerufen werden. Die Ergebnisse des Backtests werden nach der Ausführung in Form von Kennzahlen und einem Diagramm dargestellt. Die Ausführungszeit des Vorgangs wird ebenfalls gemessen und protokolliert.

Für die Performanztests wurden drei unterschiedliche Handelsstrategien mit unterschiedlicher Komplexität implementiert. Es wurden eine Gleitende Durchschnitts Überschreitungsstrategie, eine Ausbruchsstrategie und eine Mittelwertrückkehrstrategie mit Bollinger Bänder entwickelt. Jede Strategie hat eine JavaScript und eine WebAssembly Variante. Diese Strategien wurden mit dem Backtester in den Browsern Chrome und Firefox mit verschiedenen Datensätzen getestet.

Die Ergebnisse des Vergleichs zeigen, dass WebAssembly bei rechenaufwendigen Strategien schneller ist als JavaScript. Dieser Performanzvorteil ist aber nicht in allen getesteten Browsern zu erkennen. Nur in Firefox hat WebAssembly einen klaren Vorteil gegenüber JavaScript. In Chrome war JavaScript in der Ausführung immer schneller.

Einer der Gründe für die langsamere Ausführung von WebAssembly bei weniger komplexen Strategien ist der Overhead durch die manuelle Speicherverwaltung. Wenn der Vergleich ohne diesen Overhead durchgeführt wird, ist WebAssembly in Firefox bei allen getesteten Strategien schneller als JavaScript. Ein weiterer Unterschied zwischen den Programmiersprachen ist die Varianz der durchschnittlichen Ausführungszeiten. In Firefox ist die Varianz der Ausführungszeiten von WebAssembly deutlich geringer als die von JavaScript. In Chrome ist dies umgekehrt.

Zusammenfassend ist festzuhalten, dass WebAssembly einen Vorteil bei der Ausführung bieten kann und in Zukunft eine wichtige Rolle in Webanwendungen spielen wird.

6.2 Ausblick und mögliche Erweiterungen des Backtesters

Der in dieser Arbeit entwickelte Backtester bietet eine Grundlage für den Vergleich der Ausführungszeiten von WebAssembly und JavaScript und ermöglicht es Strategien mit einem Backtest zu analysieren. Der Backtester kann zukünftig erweitert und angepasst werden, um weitere Funktionalität und Auswertungsmöglichkeiten zu bieten.

Eine mögliche Erweiterung, um den realen Aktienmarkt besser nachbilden zu können, ist Transaktionskosten und Spesen ebenfalls zu berücksichtigen. Damit werden Strategien die häufige Käufe und Verkäufe durchführen realitätsnäher bewertet. Mehr Parameter für die Konfiguration des Backtesters würden noch mehr Flexibilität bieten. Es könnten dadurch vorzeitige Abbruchkriterien, Dividenden oder die Transaktionskosten in den Backtest miteinbezogen werden.

Durch das Hinzufügen eines Datumsfilters wäre es möglich, die Strategie nur in bestimmten Zeiträumen zu betrachten und die Kennzahlen für diese Zeiträume zu berechnen. Dies biete die Möglichkeit Strategien genauer zu analysieren.

Eine weitere Verbesserung wäre die Möglichkeit, mehrere Strategien und Datensätze gleichzeitig zu testen. Somit können mit einem Durchlauf mehrere Testfälle gleichzeitig und benutzerfreundlicher angewandt werden. Es müsste nicht mehr für jede Strategie und jeden Datensatz ein einzelner Test durchgeführt werden.

Um die Berechnungen und grafischen Elemente stärker zu trennen, könnte die Berechnung in einen Webworker ausgelagert werden. Somit würde die zeitaufwendige Berechnung nicht mehr die Benutzeroberfläche blockieren.

Eine Möglichkeit die Resultate als CSV oder JSON-Datei herunterzuladen würde die Weiterverarbeitung der Ergebnisse erleichtern. Durch diese Erweiterung müssten die Kennzahlen nicht mehr manuell abgelesen und in eine Tabelle übertragen werden.

Mit diesen Erweiterungen und Verbesserungen kann der Backtester weiterentwickelt werden, um einen besseren Vergleich der Strategien zu ermöglichen, den realen Handel besser zu simulieren und die Implementierung besser zu separieren.

Quellenverzeichnis

Literatur

- [1] Madhav Agarwal. *Breakout Pattern - Meaning and Trading Strategy*. Accessed: 2025-09-19. 2024. URL: <https://www.wrightresearch.in/blog/breakout-pattern-meaning-and-trading-strategy/> (siehe S. 11).
- [2] Alexey Andreev und TeaVM Contributors. *TeaVM Website*. Accessed: 2025-10-26. 2013. URL: <https://teavm.org/> (siehe S. 22).
- [3] P. Arumugam und R. Saranya. „An Empirical Analysis of Trading Strategy Based on Simple Moving Average Crossovers“. *ICTACT Journal on Management Studies* 3.1 (2017), S. 423–426 (siehe S. 11).
- [4] David H. Bailey und Marcos López de Prado. „How “Backtest Overfitting” in Finance Leads to False Discoveries“. *Significance* 18.6 (2021), S. 22–25 (siehe S. 5).
- [5] David H. Bailey u. a. „Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance“. *Notices of the American Mathematical Society* 61.5 (2014), S. 458–471 (siehe S. 6).
- [6] David H. Bailey u. a. „The Probability of Backtest Overfitting“. *Journal of Computational Finance (Risk Journals)* (2015) (siehe S. 4, 6).
- [7] Andrew Baronick. *BacktestJS Example Result*. Accessed: 2025-09-29. 2024. URL: <https://backtestjs.com/docs/trading-strategy-results/view-trading-strategy-results/> (siehe S. 9).
- [8] Andrew Baronick. *BacktestJS Website*. Accessed: 2025-09-29. 2024. URL: <https://backtestjs.com/> (siehe S. 9).
- [9] Rim El Bernoussi und Michael Rockinger. „Rebalancing with transaction costs: theory, simulations, and actual data“. *Financial Markets and Portfolio Management* 37.2 (2023), S. 121–160 (siehe S. 6).
- [10] Binance. *Binance - Cryptocurrency Exchange*. Accessed: 2025-11-03. 2025. URL: <https://www.binance.com/> (siehe S. 22, 25, 36).
- [11] John Bollinger. „Using bollinger bands“. *Stocks & Commodities* 10.2 (1992), S. 47–51 (siehe S. 10).
- [12] William Brock, Josef Lakonishok und Blake LeBaron. „Simple Technical Trading Rules and the Stochastic Properties of Stock Returns“. *The Journal of Finance* 47.5 (1992), S. 1731–1764 (siehe S. 11, 19).

- [13] Stephen J. Brown u. a. „Survivorship Bias in Performance Studies“. *The Review of Financial Studies* 5.4 (2015), S. 553–580 (siehe S. 7).
- [14] Mark M. Carhart u. a. „Mutual Fund Survivorship“. *The Review of Financial Studies* 15.5 (2015), S. 1439–1463 (siehe S. 7).
- [15] Emscripten Contributors. *Emscripten Toolchain Website*. Accessed: 2025-10-26. 2015. URL: <https://emscripten.org/> (siehe S. 23).
- [16] Emscripten Contributors. *Emscripten Website*. Accessed: 2025-10-26. 2015. URL: <https://emscripten.org/> (siehe S. 22).
- [17] Mozilla.org contributors. *Compiling from Rust to WebAssembly*. Accessed: 2025-10-26. 2025. URL: https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Rust_to_Wasm (siehe S. 22).
- [18] Gilles Daniel, Didier Sornette und Peter Wohrmann. *Look-Ahead Benchmark Bias in Portfolio Performance Evaluation*. 2008 (siehe S. 6).
- [19] Poltavskyi Dmytro. „Integration of WebAssembly in Performance-critical Web Applications“. *American Scientific Research Journal for Engineering, Technology, and Sciences* 102.1 (2025), S. 39–53 (siehe S. 13).
- [20] Fernando F. Ferreira, A. Christian Silva und Ju-Yi Yen. „Detailed Study of a Moving Average Trading Rule“. *Quantitative Finance* 18.9 (2018), S. 1599–1617 (siehe S. 11).
- [21] Luis A. Gil-Alana. „Mean reversion in the real exchange rates“. *Economics Letters* 69.3 (2000), S. 285–288 (siehe S. 10).
- [22] WebAssembly Community Group. *WebAssembly*. Accessed: 2025-09-07. 2025. URL: <https://webassembly.org/> (siehe S. 12, 14, 18).
- [23] Ikhlaas Gurrib. „The Moving Average Crossover Strategy: Does It Work for the S&P500 Market Index?“. *Global Review of Accounting and Finance* 7.1 (2016), S. 92–107 (siehe S. 11).
- [24] Andreas Haas u. a. „Bringing the web up to speed with WebAssembly“. *SIGPLAN Not.* 52.6 (2017), S. 185–200 (siehe S. 12, 14, 18).
- [25] Campbell Harvey und Yan Liu. „Backtesting“. *SSRN Electronic Journal* 42 (2013) (siehe S. 5).
- [26] David Herrera u. a. „Numerical Computing on the Web: Benchmarking for the Future“. In: *Proceedings of the 14th ACM SIGPLAN International Symposium on Dynamic Languages (DLS 2018)*. ACM, 2018, S. 88–100 (siehe S. 13).
- [27] Ulf Holmberg, Carl Lönnbark und Christian Lundström. „Assessing the profitability of intraday opening range breakout strategies“. *Finance Research Letters* 10.1 (2013), S. 27–33 (siehe S. 11).
- [28] Alexandra Horobet u. a. „Technology-driven advancements: Mapping the landscape of algorithmic trading literature“. *Technological Forecasting and Social Change* 209 (2024) (siehe S. 1).
- [29] Jing-Zhi Huang und Zhijian (James) Huang. „Testing moving average trading strategies on ETFs“. *Journal of Empirical Finance* 57 (2020), S. 16–32 (siehe S. 11).

- [30] John Hughes. „Why Functional Programming Matters“. In: *Research Topics in Functional Programming*. Hrsg. von David Turner. Addison-Wesley, 1990, S. 17–42 (siehe S. 18).
- [31] Quantopian Inc. *Zipline Trader Website*. Accessed: 2025-09-30. 2020. URL: <https://zipline-trader.readthedocs.io/en/latest/index.html> (siehe S. 8).
- [32] Quantopian Inc. und Stefan Jansen. *Zipline Reloaded Website*. Accessed: 2025-09-30. 2020. URL: <https://zipline.ml4trading.io/> (siehe S. 8).
- [33] Abhinav Jangda u. a. „Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code“. In: *2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association, 2019, S. 107–120 (siehe S. 13).
- [34] Audrius Kabasinskas und Ugnius Macys. „Calibration of Bollinger Bands Parameters for Trading Strategy Development in the Baltic Stock Market“. *Inzinerine Ekonomika-Engineering Economics* 21.3 (2010), S. 244–254 (siehe S. 19).
- [35] Mark Leeds. *Bollinger Bands Thirty Years Later*. 2013 (siehe S. 10).
- [36] Jure Leskovec und Christos Faloutsos. „Sampling from large graphs“. In: *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*. Philadelphia, PA, USA: Association for Computing Machinery, 2006, S. 631–636 (siehe S. 21).
- [37] Andrew W. Lo und A. Craig MacKinlay. *Data-snooping Biases in Tests of Financial Asset Pricing Models*. Techn. Ber. No. 3020-89-EFA. Massachusetts Institute of Technology, Sloan School of Management, 1989 (siehe S. 7).
- [38] Richard Martin und Torsten Schöneborn. *Mean Reversion Pays, but Costs*. 2011 (siehe S. 10).
- [39] Cory Mitchell. *Failed Break: What It Is, How It Works, Example*. Accessed: 2025-09-19. 2023. URL: <https://www.investopedia.com/terms/f/failedbreak.asp> (siehe S. 11).
- [40] James M. Poterba und Lawrence H. Summers. „Mean reversion in stock prices: Evidence and Implications“. *Journal of Financial Economics* 22.1 (1988), S. 27–59 (siehe S. 10).
- [41] Daniel Rodriguez. *Backtrader Website*. Accessed: 2025-09-29. 2015. URL: <https://www.backtrader.com/> (siehe S. 8).
- [42] Alan Romano und Weihang Wang. „When Function Inlining Meets WebAssembly: Counterintuitive Impacts on Runtime Performance“. In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, 2023, S. 350–362 (siehe S. 30).
- [43] *SpiderMonkey: Mozillas WebAssembly Engine*. Accessed: 2025-12-09. 2025. URL: <https://spidermonkey.dev> (siehe S. 13, 35).
- [44] *SpiderMonkey: WebAssembly Pipeline*. Accessed: 2025-12-09. 2025. URL: <https://firefox-source-docs.mozilla.org/js/index.html> (siehe S. 14).
- [45] StatCounter Global Stats. *Browser Market Share Worldwide*. Accessed: 2025-10-31. 2025. URL: <https://gs.statcounter.com/browser-market-share> (siehe S. 22).

- [46] Guanru Su. „Analysis of the Bollinger Band Mean Regression Trading Strategy“. In: *Proceedings of the 3rd International Conference on Economic Development and Business Culture (ICEDBC 2023)*. Atlantis Press, 2023, S. 95–102 (siehe S. 10).
- [47] Ryan Sullivan, Allan Timmermann und Halbert White. „Data-Snooping, Technical Trading Rule Performance, and the Bootstrap“. *The Journal of Finance* 54.5 (1999), S. 1647–1691 (siehe S. 7).
- [48] Mircea Țălu. „A Comparative Study of WebAssembly Runtimes: Performance Metrics, Integration Challenges, Application Domains, and Security Features“. *Archives of Advanced Engineering Science* 00.00 (2025), S. 1–13 (siehe S. 13).
- [49] *V8 WebAssembly Pipeline*. Accessed: 2025-12-09. 2025. URL: <https://v8.dev/docs/wasm-compilation-pipeline> (siehe S. 14).
- [50] *V8: Googles WebAssembly Engine*. Accessed: 2025-12-09. 2025. URL: <https://v8.dev> (siehe S. 13, 35).
- [51] W3Counter. *Browser & Platform Market Share – Global Web Stats*. Accessed: 2025-10-31. 2025. URL: <https://www.w3counter.com/globalstats.php> (siehe S. 22).
- [52] Yutian Yan u. a. „Understanding the performance of webassembly applications“. In: *Proceedings of the 21st ACM Internet Measurement Conference*. Association for Computing Machinery, 2021, S. 533–549 (siehe S. 13, 41).
- [53] Carlo Zarattini, Andrea Barbon und Andrew Aziz. *A Profitable Day Trading Strategy For The U.S. Equity Market*. Research Paper 24-98. Swiss Finance Institute, 2024 (siehe S. 11).
- [54] Saurabh Zunke und Veronica D’Souza. „JSON vs XML: A Comparative Performance Analysis of Data Exchange Formats“. *International Journal of Computer Science and Network* 3.4 (2014), S. 257–261 (siehe S. 16).

Software

- [55] Alexey Andreev und TeaVM Contributors. *TeaVM GitHub*. Accessed: 2025-10-26. 2013. URL: <https://github.com/konsoletyper/teavm> (siehe S. 22).
- [56] founder) BacktestingMax (Max. *BacktestingMax*. Accessed: 2025-10-18. 2025. URL: <https://backtestingmax.com> (siehe S. 7).
- [57] Andrew Baronick. *BacktestJS GitHub*. Version 1.0.0. Accessed: 2025-09-29. 2024. URL: <https://github.com/andrewbaronick/BacktestJS> (siehe S. 9).
- [58] Emscripten Contributors. *Emscripten GitHub*. Accessed: 2025-10-26. 2015. URL: <https://github.com/emscripten-core/emscripten> (siehe S. 22).
- [59] WebAssembly Community Group. *WebAssembly Binary Toolkit (WABT) GitHub*. Accessed: 2025-10-26. 2025. URL: <https://github.com/WebAssembly/wabt> (siehe S. 22, 23).
- [60] Quantopian Inc. *Quantopian PyFolio GitHub*. Version 0.9.2. Accessed: 2025-09-30. 2015. URL: <https://github.com/quantopian/pyfolio> (siehe S. 8).

- [61] Quantopian Inc. *Zipline GitHub*. Version 1.4.1. Accessed: 2025-09-30. 2020. URL: <https://github.com/quantopian/zipline> (siehe S. 8).
- [62] Quantopian Inc. und Stefan Jansen. *Zipline Reloaded GitHub*. Version 3.1.1. Accessed: 2025-09-30. 2020. URL: <https://github.com/stefan-jansen/zipline-reloaded> (siehe S. 8).
- [63] Quantopian Inc. und Shlomi Kushchi. *Zipline GitHub*. Version 1.6.0. Accessed: 2025-09-30. 2020. URL: <https://github.com/shlomiku/zipline-trader> (siehe S. 8).
- [64] Daniel Rodriguez. *Backtrader GitHub*. Version 1.9.78.123. Accessed: 2025-09-29. 2015. URL: <https://github.com/mementum/backtrader> (siehe S. 8).
- [65] Backtester.io team. *Backtester.io*. Accessed: 2025-10-18. 2025. URL: <https://backtester.io> (siehe S. 7).
- [66] FX Replay team. *FX Replay*. Accessed: 2025-10-18. 2025. URL: <https://www.fxreplay.com/> (siehe S. 7).

Abbildungsverzeichnis

2.1	Ablauf eines Backtestingsprozesses	5
2.2	Beispielerggebnis eines Backtests mit BacktestJS [7]	9
2.3	Beispiel für ein Bollinger Band	10
2.4	Vergleich von C Quellcode und generiertem WebAssembly Bytecode . .	12
2.5	Ausführung von WebAssembly im Browser	14
3.1	Struktur des Backtesters	15
3.2	Backtestingablauf	17
3.3	Funktionsschnittstellen der Strategien in C und JavaScript	18
3.4	Design der Weboberfläche des Backtesters	20
3.5	Kursdiagramme mit Kennzahlen	21
3.6	Emscripten Pipeline	23
4.1	Eingabefelder der Backtestparameter	24
5.1	Beispiel eines Kursdatensatzes im JSON-Format	36
5.2	Durchschnittliche Ausführungszeiten der Strategien in Firefox mit 1,5 Millionen Datenpunkten	38
5.3	Durchschnittliche Ausführungszeiten der Strategien in Firefox mit einer halben Million Datenpunkten	39
5.4	Standardabweichung der Strategien in Firefox	40
5.5	Durchschnittliche Ausführungszeiten der Strategien in Chrome mit 1,5 Millionen Datenpunkten	41
5.6	Durchschnittliche Ausführungszeiten der Strategien in Chrome mit einer halben Million Datenpunkten	42
5.7	Standardabweichung der Strategien in Chrome	43
5.8	Durchschnittliche Ausführungszeiten der Strategien ohne Speicherverwal- tung in Firefox mit 1,5 Millionen Datenpunkten	45
5.9	Durchschnittliche Ausführungszeiten der Strategien ohne Speicherverwal- tung in Chrome mit 1,5 Millionen Datenpunkten	46

Tabellenverzeichnis

5.1	Übersicht der Rahmenbedingungen der Performanzmessung	37
5.2	Vergleich der Ausführungszeiten von WebAssembly in den beiden Browsern	44

Listings

4.1	Aufruf der Binance API Schnittstelle	25
4.2	Laden der Strategien aus den hochgeladenen Dateien	26
4.3	Backtestlogik	27
4.4	Durchschnittsüberschneidung in C	29
4.5	Durchschnittsüberschneidung in JavaScript	30
4.6	Mittelwertrückkehr in C	31
4.7	Mittelwertrückkehr in JavaScript	31
4.8	Ausbruchsstrategie in C	32
4.9	Ausbruchsstrategie in JavaScript	32
4.10	Kurswertvisualisierung	33
4.11	Portfoliovisualisierung	34